

АННОТАЦИЯ

Выпускная квалификационная работа посвящена разработке информационной системы потоковой передачи видеоконтента. В работе проведён анализ предметной области обработки и публикации видео, рассмотрены существующие платформы видеостриминга и выявлены их ограничения с точки зрения контроля внутреннего конвейера обработки и интеграции AI-улучшения. Сформированы функциональные и нефункциональные требования к системе. Разработаны архитектура информационной системы, модель данных, пользовательские сценарии и интерфейсные решения. Спроектирован и описан конвейер обработки видеоконтента, включающий загрузку файла, валидацию, постановку задания в очередь, транскодирование в HLS-представления, генерацию миниатюр и опциональное улучшение качества с использованием Real-ESRGAN. Основным результатом работы является реализация web-системы, обеспечивающей загрузку, обработку, публикацию и просмотр видеоконтента. Практическая ценность работы заключается в возможности применения разработанной системы в образовательных, корпоративных и медиасервисах. Общий объём работы составляет 60 страниц основного текста, без приложений, 12 таблиц, 13 рисунков, 21 использованный источник. Ключевые слова: видеоконтент, видеостриминг, потоковая передача, HLS, VOD, Real-ESRGAN, FFmpeg, web-система, SPA, Go, PostgreSQL, Redis, Vue 3, обработка видео.

ПЕРЕЧЕНЬ СОКРАЩЕНИЙ И ОБОЗНАЧЕНИЙ

API – интерфейс программирования приложений;

CDN – сеть доставки контента;

GPU – графический процессор;

HLS – HTTP Live Streaming;

HTTP – протокол передачи гипертекста;

JSON – текстовый формат обмена данными;

REST – архитектурный стиль построения web-API;

SPA – одностраничное web-приложение;

SQL – язык структурированных запросов;

UI – пользовательский интерфейс;

VOD – модель доставки видео по запросу;

СОДЕРЖАНИЕ

ПЕРЕЧЕНЬ СОКРАЩЕНИЙ И ОБОЗНАЧЕНИЙ	3
ВВЕДЕНИЕ	4
1 Описание предметной области.....	7
1.1 Особенности обработки и потоковой передачи видеоконтента	7
1.2 Описание бизнес-процессов в предметной области	10
1.3 Анализ существующих информационных систем и платформ для видеостриминга	13
1.4 Функциональные и нефункциональные требования к информационной системе	16
1.5 Обоснование выбора средств разработки информационной системы	18
Выводы по первой главе.....	24
2 Проектирование информационной системы потоковой передачи видеоконтента	27
2.1 Моделирование информационной системы в нотации UML	27
2.1.1 Диаграмма вариантов использования.....	28
2.1.2 Диаграмма деятельности	31
2.1.3 Диаграммы последовательностей	34
2.1.4 Диаграмма классов	36
2.1.5 Диаграмма компонентов.....	39
2.1.6 Диаграмма развертывания	40
2.2 Проектирование базы данных информационной системы.....	42
2.3 Проектирование интерфейса информационной системы.....	51
2.4 Описание алгоритмов функционирования информационной системы	54
2.5 Архитектура информационной системы.....	57
Выводы по второй главе	60
3 Разработка информационной системы потоковой передачи видеоконтента	62
3.1 Программная реализация базы данных и серверных модулей	62
3.2 Реализация клиентской части и пользовательских сценариев.....	65
3.3 Реализация конвейера обработки видеоконтента.....	68
3.4 Тестирование и оценка работоспособности системы	70
Выводы по третьей главе.....	72
ЗАКЛЮЧЕНИЕ	74
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	76

ВВЕДЕНИЕ

Видеоконтент стал одним из базовых форматов представления информации в образовательных, медийных, корпоративных и сервисных цифровых платформах. Для пользователя современного веб-сервиса важны не только наличие самого видеофайла, но и удобство его загрузки, устойчивость воспроизведения, адаптация качества под пропускную способность канала, корректная работа на разных устройствах и возможность последующей обработки материала. В результате разработка информационных систем, ориентированных на полный жизненный цикл видеоданных, перестаёт быть узкой задачей мультимедийных платформ и становится частью общей практики проектирования прикладных веб-сервисов.

При работе с видеоматериалами возникает сразу несколько классов задач. К ним относятся приём и проверка исходного файла, извлечение технических характеристик, хранение метаданных, постановка ресурсоёмких операций в очередь, транскодирование в набор потоков с различным битрейтом, упаковка сегментов для адаптивного воспроизведения и организация пользовательского интерфейса для публикации и просмотра. Дополнительную сложность создаёт необходимость улучшения качества отдельных материалов, поскольку исходные записи могут быть зашумлены, сжаты с потерями либо получены в низком разрешении. В таких условиях требуется система, которая объединяет средства потоковой доставки и средства интеллектуальной постобработки в едином управляемом конвейере.

Актуальность темы обусловлена тем, что существующие публичные видеоплатформы ориентированы прежде всего на массовое размещение контента и не всегда обеспечивают достаточный уровень контроля над внутренним процессом обработки, политикой хранения данных, порядком публикации, интеграцией с внутренними сервисами и применением собственных алгоритмов

улучшения качества. Для прикладных систем, предназначенных для образовательных порталов, закрытых медиатек, корпоративных баз знаний и специализированных видеосервисов, более целесообразным является построение собственной информационной системы, в которой логика загрузки, обработки и показа видеоконтента определяется требованиями конкретного заказчика.

Целью выпускной квалификационной работы является повышение удобства и эффективности обработки видеоконтента за счёт создания веб-системы для его загрузки и улучшения качества.

Для достижения поставленной цели необходимо решить следующие задачи:

- 1 проанализировать предметную область обработки и потоковой передачи видеоконтента;
- 2 определить требования к разрабатываемой веб-системе;
- 3 разработать архитектуру информационной системы;
- 4 спроектировать механизм загрузки, хранения и обработки видеоконтента;
- 5 реализовать основные модули веб-системы;
- 6 провести тестирование и оценить работоспособность системы.

Объектом исследования являются процессы загрузки, хранения, обработки и потоковой доставки видеоконтента в веб-среде.

Предметом исследования являются методы и программные средства построения информационной системы, обеспечивающей приём видеоматериалов, асинхронную обработку, формирование потоковых представлений и улучшение качества видео с использованием алгоритмов суперразрешения.

В работе используются методы системного анализа, сравнительного анализа программных платформ, проектирования информационных систем, моделирования бизнес-процессов, а также практические методы разработки веб-приложений и тестирования программного обеспечения.

Практическая значимость работы заключается в возможности применения разрабатываемой системы для построения корпоративных видеоплатформ, закрытых библиотек медиаматериалов, образовательных порталов и сервисов, где требуется контролируемый цикл публикации и обработки видеофайлов. Дополнительной особенностью проекта является включение в общий конвейер модуля улучшения качества, который позволяет повысить полезность уже имеющегося контента без переноса данных в сторонние сервисы.

1 Описание предметной области

1.1 Особенности обработки и потоковой передачи видеоконтента

Предметная область разрабатываемой системы связана с обработкой видеоданных по модели video-on-demand, при которой пользователь загружает исходный файл, после чего система выполняет подготовку материалов к дальнейшему воспроизведению в веб-плеере. В отличие от традиционного хранения файлов на общем сетевом ресурсе, здесь видео рассматривается как сложный объект, включающий двоичные данные, набор технических характеристик, производные представления разных качеств, служебные артефакты обработки и пользовательские метаданные. Такой подход позволяет разделить задачи хранения исходника, отображения каталога, управления доступом и потоковой выдачи готовых сегментов.

Видеофайл обладает набором признаков, которые непосредственно влияют на процесс его публикации. К ним относятся контейнер, кодек видеоряда, аудиокодек, частота кадров, разрешение, длительность, битрейт, формат цветового пространства и наличие вспомогательных дорожек. Даже если пользователю кажется, что операция загрузки сводится к передаче одного файла, с точки зрения системы уже на начальном этапе возникает необходимость валидации формата, нормализации имён, расчёта контрольных сумм, извлечения медиаметаданных и проверки ограничений, установленных политикой сервиса. В условиях промышленной эксплуатации эти действия должны выполняться автоматически и воспроизводимо, иначе вероятность ошибок при подготовке публикации существенно возрастает.

Для веб-доставки видеоконтента применяется несколько подходов. Самым простым является прогрессивная передача файла по HTTP, при которой клиент получает единый ресурс и воспроизводит его по мере загрузки. Такой вариант удобен в реализации, однако плохо приспособлен к изменяющимся сетевым

условиям и не позволяет гибко переключаться между потоками разного качества. Поэтому на практике значительно шире используются адаптивные протоколы потоковой передачи, прежде всего HLS и MPEG-DASH. В RFC 8216 указывается, что HLS обеспечивает адаптацию битрейта под текущие сетевые условия и тем самым помогает поддерживать непрерывное воспроизведение наилучшего возможного качества [1]. Аналогичную задачу решает и MPEG-DASH, развиваемый как открытый стандарт экосистемой DASH Industry Forum [2].

С точки зрения проектирования информационной системы важно не только выбрать формат конечной выдачи, но и организовать внутренний конвейер подготовки потоков. Обычно исходное видео преобразуется в несколько представлений: например, 360p, 480p, 720p и 1080p, каждое из которых имеет собственный битрейт и параметры кодирования. Далее формируются сегменты фиксированной длины и манифест, содержащий сведения о доступных вариантах. Браузерный или встроенный плеер, анализируя скорость сети и буфер, выбирает подходящее представление. Если система не поддерживает такой механизм, то пользователь либо получает избыточно тяжёлый поток и сталкивается с буферизацией, либо вынужден смотреть материал в искусственно заниженном качестве.

Задача транскодирования является ключевой для всей предметной области. Она объединяет декодирование исходного медиапотока, изменение кодека или контейнера, нормализацию аудио, масштабирование изображения, вставку ключевых кадров, генерацию миниатюр и упаковку сегментов. В проектируемой системе данная функция должна выполняться средствами FFmpeg, который предоставляет единый набор инструментов для работы с форматами, кодеками, фильтрами и потоковыми протоколами [3]. Использование такого инструмента позволяет строить воспроизводимый конвейер из командных профилей и автоматически запускать его в фоновом режиме без привязки к конкретному поставщику облачных медиасервисов.

Отдельного внимания заслуживает проблема качества исходного материала. Во многих предметных областях — например, в корпоративном обучении, внутренних архивах записей, трансляциях вебинаров и пользовательском контенте — в систему поступают ролики, снятые на разные устройства, с различным уровнем освещённости, сжатием и исходным разрешением. Если публиковать такие материалы без дополнительной обработки, то итоговое впечатление от сервиса и ценность накопленной медиабазы снижаются. В связи с этим в состав системы целесообразно включить опциональный модуль улучшения качества. В работе предлагается использовать Real-ESRGAN, ориентированный на восстановление и суперразрешение изображений в условиях реальных и сложных искажений [11]. Практическая интеграция может быть выполнена через portable-реализацию Real-ESRGAN-ncnn-vulkan, рассчитанную на запуск без тяжёлого Python-окружения и использующую графические ускорители через Vulkan [12].

Важно подчеркнуть, что применение алгоритмов суперразрешения к видеоконтенту должно рассматриваться как специальный, а не обязательный этап обработки. По сравнению с обычным транскодированием, AI-улучшение существенно более ресурсоёмко, требует тщательного выбора профиля обработки и не всегда оправдано для каждого ролика. Поэтому архитектурно оно должно быть встроено в систему как отдельная ветка конвейера, активируемая по запросу пользователя или по административному правилу. Такая организация позволяет сохранить управляемость нагрузки и не блокировать базовые сценарии публикации.

Ещё одной особенностью предметной области является разделение собственно медиаданных и прикладной информации о публикации. Бинарные файлы, сегменты и миниатюры обычно хранятся в файловом либо объектном хранилище, тогда как сведения о пользователях, правах доступа, названиях, описаниях, тегах, статусах обработки и истории операций целесообразно

поддерживать в реляционной базе данных [20], [21]. Это даёт возможность независимо масштабировать хранение файлов и прикладной контур, а также выполнять сложные выборки и фильтрацию по каталогу видеоматериалов.

Таким образом, предметная область потоковой передачи видеоконтента объединяет задачи веб-разработки, распределённой обработки, мультимедийного кодирования и управления данными. Для успешной реализации системы необходимо обеспечить согласованную работу пользовательского интерфейса, серверного API, очередей фоновых задач, средств транскодирования и хранилища метаданных. Именно эта совокупность требований определяет последующий выбор архитектурных решений и технологического стека.

Таблица 1.1 – Сравнение основных способов доставки видеоконтента

Способ доставки	Преимущества	Ограничения	Целесообразность для проекта
Прогрессивная загрузка	Простая реализация, минимальные требования к серверной логике	Нет адаптации качества, неэффективна при нестабильной сети	Подходит только для простых архивных сценариев
HLS	Широкая поддержка клиентов, адаптивный битрейт, удобная сегментация	Необходим этап упаковки сегментов и формирование плейлиста	Основной механизм публикации для веб-плеера
MPEG-DASH	Открытый стандарт, гибкая модель представления потоков	Поддержка в экосистеме браузеров зависит от плеера и платформы	Перспективное направление расширения системы

1.2 Описание бизнес-процессов в предметной области

Для разработки информационной системы недостаточно описать только технические свойства видеофайлов; необходимо определить, какие бизнес-процессы будут поддерживаться системой и как распределяются действия между участниками. В общем виде можно выделить три основные роли: администратор, оператор (или контент-менеджер) и пользователь, просматривающий опубликованные материалы. Каждая из ролей работает с одной и той же

видеобазой, но использует разные функции интерфейса и разные точки входа в систему.

Наиболее характерная проблема существующих, неавтоматизированных сценариев состоит в фрагментарности операций. Видео может загружаться в одну систему, конвертироваться вручную локальными инструментами, описываться в электронных таблицах, а публиковаться уже на сторонней платформе. При таком подходе затрудняется отслеживание статуса обработки, появляются дубликаты файлов, сложно контролировать версии и фактическое качество опубликованного материала. Кроме того, любые дополнительные операции — например, повторное транскодирование или улучшение качества — требуют ручного участия и фактически выводят процесс за пределы единой информационной системы.

Проектируемая система должна устранить указанные недостатки за счёт формализации и автоматизации ключевых шагов. Бизнес-процесс начинается с авторизации пользователя и выбора сценария загрузки. После передачи исходного файла в веб-интерфейсе формируется карточка видео, в которой фиксируются название, описание, теги, категория, владелец, политика доступа и статус публикации. Далее сервер принимает файл, сохраняет информацию о нём в базе данных и передаёт задание в очередь фоновых операций. Наличие отдельной очереди принципиально важно: оно позволяет не удерживать HTTP-запрос пользователя открытым на протяжении всего транскодирования и в то же время даёт возможность наблюдать ход обработки через систему статусов.

Следующим этапом становится технический анализ исходного материала. Worker-процесс извлекает медиаметаданные, определяет длительность, частоту кадров, разрешение, параметры аудио и подготавливает набор операций, соответствующий выбранному профилю публикации. После этого запускается FFmpeg-конвейер: создаются представления разных разрешений, формируются сегменты HLS, генерируются миниатюры и при необходимости выделяются

кадры для последующего улучшения качества. Если пользователь активировал интеллектуальное улучшение, задача транскодирования дополняется стадией обработки через Real-ESRGAN-ncnn-vulkan; при этом готовые данные вновь возвращаются в общий поток публикации.

После завершения обработки система переводит запись видео в состояние готовности, связывает карточку ролика с манифестом потоковой выдачи и открывает материал для просмотра либо оставляет его в режиме черновика. Для конечного пользователя бизнес-процесс выглядит проще: он обращается к каталогу, выбирает видео, получает в плеер адрес HLS-манифеста и запускает воспроизведение. Однако внутри системы в этот момент также происходят прикладные операции: регистрируется факт просмотра, ведётся журнал обращений, а плеер адаптирует качество под доступную пропускную способность канала.

Немаловажной частью предметной области является обработка нештатных ситуаций. Видео может быть загружено в неподдерживаемом формате, прерваться на этапе передачи файла, завершиться ошибкой кодирования или не пройти AI-улучшение из-за нехватки графической памяти. Поэтому бизнес-процесс должен предусматривать состояния ошибки, механизмы повторного запуска задачи, уведомления оператора и сохранение технических логов. С точки зрения проектирования это означает, что каждая фоновая операция должна быть идемпотентной настолько, насколько это возможно, а переходы между состояниями — явно фиксироваться в базе данных.

В результате предметная область описывается не одной операцией загрузки, а совокупностью взаимосвязанных процессов: управления пользователями, управления каталогом видео, фоновой обработки, публикации, просмотра и администрирования. Такое представление важно для дальнейшего проектирования, поскольку позволяет выделить сущности базы данных, набор

REST-эндпоинтов, требования к очередям задач и принципы декомпозиции интерфейса.

Таблица 1.2 – Основные роли пользователей системы

Роль	Основные функции	Ограничения ответственности
Администратор	Управление пользователями, правами доступа, системными настройками, повторным запуском задач и аудитом операций	Не выполняет повседневную подготовку контента, а контролирует инфраструктурный и организационный контур
Оператор контента	Загрузка видео, заполнение метаданных, запуск обработки, контроль статусов, публикация и архивирование материалов	Работает в рамках выделенного каталога и не изменяет глобальные настройки системы
Пользователь	Поиск видео, просмотр потока, использование каталога и страниц публикаций	Не имеет доступа к процессам загрузки, кодирования и администрирования

Таблица 1.3 – Основные бизнес-процессы разрабатываемой системы

Процесс	Входные данные	Результат
Авторизация и разграничение прав	Учётные данные пользователя, роль, политика доступа	Предоставление доступа к разрешённым функциям интерфейса и API
Загрузка видео и ввод метаданных	Исходный файл, название, описание, теги, правила публикации	Создание карточки видео и сохранение оригинала в хранилище
Постановка и выполнение фоновой обработки	Идентификатор видео, профиль кодирования, параметры AI-улучшения	Готовые представления разных качеств, сегменты, превью и статусы выполнения
Публикация материала	Готовый манифест, карточка видео, политика видимости	Доступное для просмотра видео или черновик публикации
Просмотр и адаптивное воспроизведение	Запрос пользователя, адрес манифеста, условия сети	Воспроизведение видео с выбором оптимального качества
Контроль ошибок и повторный запуск задач	Журналы обработки, код ошибки, история запуска	Восстановление обработки без повторной загрузки оригинала

1.3 Анализ существующих информационных систем и платформ для видеостриминга

Анализ существующих платформ необходим для того, чтобы определить, какие возможности уже представлены на рынке и какие ограничения

сохраняются при использовании готовых решений. Для рассматриваемой предметной области показательное сравнение массовых видеоплатформ, коммерческих B2B-сервисов и self-hosted решений. Такое сопоставление позволяет отделить универсальные функции — загрузку, перекодирование, плеер и каталог — от специальных требований проекта, связанных с управляемым конвейером обработки и интеграцией AI-улучшения.

YouTube является наиболее известной массовой платформой публикации видео. Сервис позволяет загружать ролики через YouTube Studio, автоматически выполняет их обработку и доставляет контент через собственную инфраструктуру [13]. С точки зрения прикладного проекта это подтверждает жизнеспособность модели, при которой пользователь работает с исходным файлом, а система самостоятельно формирует потоковые представления. Однако массовый характер платформы означает и ряд ограничений: логика обработки закрыта, интеграция с внутренними системами ограничена рамками внешнего API, а политика размещения определяется правилами поставщика сервиса.

Vimeo ориентирован в большей степени на профессиональное и корпоративное использование видео. Платформа предоставляет инструменты загрузки, встраивания, настройки приватности и работы через API; отдельные руководства посвящены программной загрузке видеоматериалов [14]. Преимуществами Vimeo выступают зрелый плеер, развитые настройки доступа и наличие разработческого контура. Вместе с тем система остаётся SaaS-платформой, поэтому заказчик зависит от тарифной политики, ограничений внешнего API и закрытого внутреннего конвейера обработки. Для задач, где требуется точный контроль над всеми этапами подготовки ролика или интеграция собственных алгоритмов обработки, этого оказывается недостаточно.

Kinescope позиционируется как видеоплатформа для бизнеса и образовательных проектов. Официальные материалы сервиса акцентируют внимание на защищённом хранении, потоковой выдаче, аналитике,

низколатентных сценариях, API и собственном CDN [17], [18]. Для организаций, которым нужен быстрый запуск корпоративного видеохостинга без самостоятельной разработки медиаконвейера, это привлекательный вариант. Тем не менее данное решение также предполагает зависимость от внешней инфраструктуры и набора возможностей, заданных вендором. При необходимости встроить специфическую очередь задач, собственную модель метаданных или отдельный этап AI-улучшения проекта придётся либо ограничиваться доступными расширениями, либо отказываться от части сценариев.

PeerTube представляет противоположный подход. Это свободная self-hosted платформа, позволяющая поднять собственный видеосервис и сохранить контроль над инфраструктурой [15]. Документация PeerTube описывает архитектуру, включающую reverse proxy, API-сервер, PostgreSQL и контур фоновых операций [16]. С этой точки зрения PeerTube ближе к задачам проектируемой системы, поскольку демонстрирует практику отделения прикладного API, хранилища метаданных и асинхронной обработки. Однако PeerTube является полнофункциональной платформой общего назначения: в ней присутствуют механизмы федерации, собственная логика публикации, социальные функции и иные компоненты, которые могут быть избыточны для специализированного сервиса потоковой передачи с управляемым конвейером подготовки контента.

Следовательно, готовые платформы хорошо решают базовые задачи публикации и воспроизведения, но каждая из них накладывает собственные рамки. Массовые сервисы минимизируют усилия на старте, но плохо подходят для интеграции нестандартной обработки. Коммерческие B2B-платформы дают больше корпоративных функций, но привязывают заказчика к внешнему поставщику. Self-hosted решения обеспечивают независимость, но часто оказываются слишком общими и требуют адаптации под конкретный сценарий.

Для темы настоящей работы наиболее рациональным является создание собственной информационной системы, в которой можно одновременно реализовать прикладную бизнес-логику, асинхронную обработку и отдельный модуль улучшения качества видео.

Таблица 1.4 – Сравнительный анализ существующих платформ видеостриминга

Критерий	YouTube	Vimeo	Kinescope	PeerTube	Разрабатываемая система
Программная загрузка контента	Да	Да	Да	Да	Да
Контроль внутреннего конвейера обработки	Нет	Частично	Частично	Частично	Да
Self-hosted развёртывание	Нет	Нет	Нет	Да	Да
Адаптивная потоковая передача	Да	Да	Да	Да	Да
Гибкая интеграция AI-улучшения	Нет	Нет	Ограничено	Требуется доработка	Да
Полный контроль над моделью данных	Нет	Нет	Нет	Частично	Да
Зависимость от внешнего вендора	Высокая	Высокая	Высокая	Низкая	Отсутствует

1.4 Функциональные и нефункциональные требования к информационной системе

Результаты анализа предметной области и существующих платформ позволяют сформулировать требования к разрабатываемой информационной системе. Требования должны охватывать не только набор пользовательских функций, но и характеристики, определяющие пригодность решения к эксплуатации: производительность, надёжность, безопасность, удобство сопровождения и масштабируемость. Для рассматриваемого проекта особенно важно разделять интерактивные сценарии, выполняемые в web-интерфейсе, и ресурсоёмкие задачи подготовки видео, которые должны выноситься в фоновый контур.

Функциональные требования описывают, какие действия система обязана поддерживать на уровне пользовательских ролей и внутренних процессов. В рамках темы ВКР ключевыми являются загрузка и описание видеоконтента, запуск асинхронной обработки, формирование потокового представления, публикация и просмотр подготовленного материала. Дополнительно требуется предусмотреть управление состояниями задач и возможность повторного запуска неудачных операций без повторной загрузки исходного файла.

Нефункциональные требования формируют качественный профиль решения. Поскольку система взаимодействует с крупными бинарными файлами и длительными операциями кодирования, для неё принципиальны отказоустойчивость фоновых задач, сохранность метаданных, возможность горизонтального масштабирования worker-процессов и удобство мониторинга. Отдельного внимания требуют безопасность загрузки файлов, разграничение прав доступа и защита административных функций от несанкционированного использования. С практической точки зрения именно нефункциональные требования определяют, сможет ли созданная система использоваться в реальных сценариях, а не только демонстрировать корректность отдельных функций.

Таблица 1.5 – Функциональные требования к информационной системе

Код	Функциональное требование	Назначение
F1	Авторизация пользователей и разграничение ролей	Разделение административных, операторских и пользовательских функций
F2	Загрузка исходных видеофайлов через web-интерфейс	Приём контента без использования сторонних платформ
F3	Валидация формата, размера и базовых метаданных файла	Предотвращение некорректной постановки задач в обработку
F4	Хранение карточки видео: название, описание, теги, статус, владелец	Организация каталога и поиск по публикациям
F5	Постановка заданий на обработку в фоновую очередь	Разделение интерактивных и длительных операций
F6	Транскодирование видео в несколько представлений качества	Подготовка адаптивного потокового воспроизведения

Код	Функциональное требование	Назначение
F7	Генерация HLS-сегментов, плейлиста и миниатюр	Публикация видео в браузерном плеере
F8	Опциональный запуск AI-улучшения качества видео	Повышение полезности низкокачественных материалов
F9	Просмотр статуса обработки и журналов выполнения задач	Контроль жизненного цикла публикации
F10	Публикация, снятие с публикации и архивирование видео	Управление доступностью контента в каталоге
F11	Поиск и фильтрация видео в пользовательском каталоге	Упрощение навигации по медиабазе
F12	Воспроизведение опубликованного видео в web-плеере	Конечный пользовательский сценарий работы с системой

Таблица 1.6 – Нефункциональные требования к информационной системе

Группа требований	Содержание требования
Производительность	Операции интерфейса каталога и административных страниц должны выполняться без заметных задержек; длительные операции кодирования должны исполняться асинхронно.
Масштабируемость	Архитектура должна допускать независимое масштабирование API-сервиса и worker-процессов обработки видео.
Надёжность	Система должна сохранять состояние задач и обеспечивать повторный запуск обработки после сбоев без повторной загрузки оригинала.
Безопасность	Необходимо разграничение прав, валидация загружаемых файлов, защита административных функций и журналирование критичных операций.
Сопровождаемость	Компоненты решения должны быть разделены по ответственности, иметь понятные интерфейсы и поддерживать расширение функционала.
Совместимость	Пользовательская часть должна корректно работать в современных браузерах настольных и мобильных устройств.
Наблюдаемость	Система должна предоставлять журналы, статусы задач и сведения о технических параметрах обработанных материалов.
Портируемость	Развёртывание решения должно быть возможно на стандартной серверной инфраструктуре без привязки к одной платформе.

1.5 Обоснование выбора средств разработки информационной системы

На основании сформулированных требований можно перейти к обоснованию технологического стека. Так как система должна сочетать web-

интерфейс, REST API, реляционное хранение метаданных и асинхронный конвейер видеобработки, целесообразно использовать модульную архитектуру с независимым клиентским приложением, серверной частью и worker-процессами. Такой подход соответствует общим принципам построения современных приложений с интенсивной обработкой данных и позволяет масштабировать различные части системы независимо друг от друга [20].

В качестве серверного языка и платформы рассматриваются несколько типичных альтернатив: Go с фреймворком Gin, Node.js с серверным фреймворком уровня NestJS/Express и Python с FastAPI. Для проекта, в котором серверный контур должен быстро обрабатывать HTTP-запросы, координировать фоновую обработку и эффективно работать с конкурентными операциями, особенно важны производительность, простота развёртывания и предсказуемость потребления ресурсов. Документация языка Go подчёркивает его ориентацию на создание безопасных и масштабируемых систем, а также удобство использования встроенных механизмов конкурентности [4]. Gin, в свою очередь, позиционируется как высокопроизводительный web-фреймворк для построения API и микросервисов [5].

Для хранения метаданных видеоматериалов, состояний задач, сведений о пользователях и журналов операций требуется реляционная СУБД с поддержкой транзакций, ограничений целостности, индексирования и сложных выборок. PostgreSQL официально описывается как надёжная объектно-реляционная СУБД с развитой функциональностью [6]. В контексте проекта её преимуществом также является поддержка JSON-полей, полнотекстового поиска, расширений и хорошая пригодность как для строго структурированных данных, так и для хранения дополнительных параметров обработки. В отличие от документоориентированных систем, выбор PostgreSQL позволяет сохранить строгую схему данных в тех областях, где важны статусы, связи и непротиворечивость каталога [21].

Для организации фоновых задач целесообразно использовать отдельный слой очередей. При сравнении Redis, RabbitMQ и Kafka следует учитывать, что в данной ВКР требуется прежде всего оперативная постановка и получение job-задач на кодирование, а не сложная маршрутизация сообщений или построение полноценной event-streaming платформы. Документация Redis подчёркивает наличие развитых структур данных, потоков и средств работы в режиме реального времени [7]. В условиях небольшого или среднего по масштабу видеосервиса Redis обеспечивает достаточную производительность и простоту эксплуатации, тогда как внедрение RabbitMQ или Kafka оправдано в более тяжёлых сценариях межсервисного обмена.

Обработка видеоданных в работе базируется на FFmpeg. Данный инструмент выбран благодаря широкой поддержке контейнеров, кодеков, фильтров, устройствам ввода-вывода и потоковым протоколам [3]. Практическое достоинство FFmpeg состоит в том, что он позволяет описать медиаконвейер декларативно — в виде набора команд и профилей кодирования. Это особенно важно для серверной интеграции: backend может формировать параметры задачи, worker — выполнять однотипные профили, а система мониторинга — сохранять журналы выполнения. В качестве альтернативы можно рассматривать GStreamer и другие мультимедийные фреймворки, однако для задач типового транскодирования и HLS-упаковки FFmpeg обеспечивает более прямолинейную и воспроизводимую интеграцию.

Для интеллектуального улучшения качества выбран инструмент Real-ESRGAN-ncnn-vulkan. С одной стороны, научная основа решения определяется работой Real-ESRGAN, где предлагается практический подход к восстановлению изображений в условиях реальных искажений [11]. С другой стороны, портативная реализация на базе NCNN и Vulkan позволяет встроить алгоритм в серверный конвейер без необходимости поддерживать отдельную Python-инфраструктуру и CUDA-зависимости [12]. Это особенно удобно в проектах, где

AI-обработка рассматривается как дополнительный этап pipeline, запускаемый на части контента или по запросу оператора.

Клиентская часть системы должна быть достаточно лёгкой для сопровождения, обеспечивать типобезопасную работу с данными и быстрое обновление интерфейса при изменении состояний задач. Vue 3 позиционируется как прогрессивный фреймворк для построения пользовательских интерфейсов [8]. TypeScript расширяет JavaScript системой статической типизации и улучшает качество сопровождения кода [9]. Vite обеспечивает современную среду разработки с быстрым dev-server и оптимизированной production-сборкой [10]. В совокупности такой стек позволяет реализовать SPA-интерфейс каталога, административных страниц и экранов мониторинга без чрезмерной архитектурной тяжеловесности.

Следовательно, выбранный стек не является случайным набором популярных инструментов. Каждый компонент закрывает конкретный блок требований: Go и Gin — API и оркестрацию, PostgreSQL — хранение и целостность данных, Redis — очередь фоновых задач, FFmpeg — медиаконвейер, Real-ESRGAN-ncnn-vulkan — AI-улучшение качества, Vue 3 с TypeScript и Vite — пользовательский интерфейс. Такое сочетание технологий обеспечивает баланс между производительностью, контролируемостью и удобством развития проекта.

Таблица 1.7 – Экспертное сравнение серверных технологий

Критерий	Go + Gin	Node.js + NestJS/Express	Python + FastAPI
Производительность HTTP-контуров	5	4	3
Удобство конкурентной обработки	5	4	3
Простота развёртывания	5	4	4
Строгость типизации и предсказуемость кода	4	4	4
Ресурсоэффективность	5	3	3
Итоговая экспертная оценка	24	19	17

Таблица 1.8 – Сравнение вариантов СУБД для проекта

Критерий	PostgreSQL	MySQL	MongoDB
Транзакционность и ограничения целостности	5	4	2
Сложные выборки и SQL-аналитика	5	4	2
Поддержка полуструктурированных данных	4	3	5
Расширяемость и индексирование	5	4	3
Уместность для каталога видео и статусов задач	5	4	3
Итоговая экспертная оценка	24	19	15

Таблица 1.9 – Сравнение средств организации фоновых задач

Критерий	Redis	RabbitMQ	Kafka
Простота внедрения в web-проект	5	4	2
Подходящесть для job-очередей	5	4	3
Операционные затраты на сопровождение	5	3	2
Гибкость для задержанных и повторных задач	4	5	3
Избыточность для масштаба ВКР	Низкая	Средняя	Высокая
Вывод	Оптимальный выбор	Допустим при сложной маршрутизации	Избыточен для базового сценария

Таблица 1.10 – Сравнение технологий клиентской части

Критерий	Vue 3 + TypeScript + Vite	React + TypeScript + Vite	Angular
Порог входа и объём шаблонного кода	Низкий	Средний	Высокий
Удобство построения SPA-интерфейса	Высокое	Высокое	Высокое
Прозрачность компонентной структуры	Высокая	Высокая	Средняя
Типобезопасность	Высокая	Высокая	Высокая
Скорость разработки и сборки	Высокая	Высокая	Средняя
Вывод	Оптимальный баланс простоты и масштабируемости	Хороший вариант, но требует больше шаблонных решений	Подходит для крупных enterprise-приложений

Критерий	Vue 3 + TypeScript + Vite	React + TypeScript + Vite	Angular
			, но избыточен

Таблица 1.11 – Итоговый выбранный стек проекта

Компонент	Выбранная технология	Обоснование выбора
Backend	Go + Gin	Высокая производительность API, удобная конкурентность, простое развёртывание и предсказуемое потребление ресурсов
База данных	PostgreSQL	Надёжное хранение метаданных, транзакции, ограничения целостности, удобство сложных запросов и фильтрации
Очередь задач	Redis	Быстрая постановка и получение job-задач, достаточность для фоновой обработки видео, простота эксплуатации
Видеообработка	FFmpeg	Поддержка широкого набора кодеков и форматов, удобство HLS-упаковки и автоматизации профилей кодирования
AI-улучшение	Real-ESRGAN-ncnn-vulkan	Портативное GPU-ускоренное улучшение качества без тяжёлого Python-окружения
Frontend	Vue 3 + TypeScript + Vite	Компонентный SPA-интерфейс, типобезопасность, высокая скорость разработки и сборки

Для оценки согласованности выбранного стека важно рассмотреть не только характеристики отдельных технологий, но и распределение ответственности между компонентами системы. Серверная часть должна решать задачи аутентификации, валидации, работы с метаданными и постановки фоновых операций, тогда как worker-контур — выполнять тяжёлую медиаподготовку без блокировки HTTP-запросов. Клиентская часть, в свою очередь, должна обеспечивать прозрачное отображение состояний публикации и задач обработки. Такое разделение обязанностей снижает связанность компонентов и позволяет масштабировать узкие места независимо друг от друга.

С точки зрения сопровождения выбранная архитектурная композиция удобна ещё и тем, что каждая подсистема может развиваться эволюционно. Например, увеличение числа worker-процессов не требует переработки клиентского интерфейса, а изменение модели метаданных в PostgreSQL не влечёт замены инструментов транскодирования. В результате проект получает не только работоспособный MVP, но и основу для дальнейшего расширения: добавления новых профилей кодирования, модулей аналитики, политик доступа и сценариев автоматической обработки контента.

Таблица 1.12 – Распределение ответственности между компонентами системы

Компонент	Зона ответственности	Основные интеграции
Vue 3 SPA	Формы загрузки, каталог, страницы публикаций, отображение статусов обработки	REST API backend, HLS-плеер
Go API	Аутентификация, валидация входных данных, работа с метаданными, выдача служебных эндпоинтов	PostgreSQL, Redis, файловое хранилище
Worker обработки	Запуск FFmpeg, генерация сегментов, миниатюр, контроль фоновых задач	Redis, файловое хранилище, PostgreSQL
AI worker	Опциональное улучшение качества через Real-ESRGAN-ncnn-vulkan	Redis, GPU/Vulkan, файловое хранилище
PostgreSQL	Хранение пользователей, карточек видео, статусов, журналов и конфигурации	Go API, worker-процессы
Redis	Очередь фоновых задач и координация обработки	Go API, worker-процессы

Выводы по первой главе

В первой главе были рассмотрены особенности предметной области обработки и потоковой передачи видеоконтента, описан жизненный цикл видеоматериала от загрузки до просмотра и выделены процессы, которые должны поддерживаться разрабатываемой системой. Проведённый анализ показал, что для данного класса задач недостаточно только хранить исходные

файлы; необходимо организовать управляемый конвейер валидации, асинхронной обработки, публикации и адаптивного воспроизведения.

Сравнение существующих платформ видеостриминга показало, что готовые решения предоставляют важные базовые возможности, но не обеспечивают полного контроля над внутренней логикой обработки и не ориентированы на интеграцию специализированного этапа AI-улучшения в требуемом объеме. По этой причине обоснован выбор собственной информационной системы, ориентированной на прикладной сценарий загрузки, подготовки и публикации видео.

На основании анализа были сформулированы функциональные и нефункциональные требования, а также выбран технологический стек проекта: Go + Gin для серверной части, PostgreSQL для хранения метаданных, Redis для организации очереди фоновых задач, FFmpeg для медиаконвейера, Real-ESRGAN-ncnn-vulkan для опционального улучшения качества и Vue 3 + TypeScript + Vite для клиентского интерфейса. Полученные результаты служат основанием для перехода к проектированию архитектуры и модели данных системы в следующих разделах выпускной квалификационной работы.

2 Проектирование информационной системы потоковой передачи видеоконтента

На основании результатов первой главы выполняется переход от исследования предметной области к проектированию целевой информационной системы потоковой передачи видеоконтента. На этом этапе формируется модель TO-BE, в которой уточняются пользовательские сценарии, структура данных, логика взаимодействия модулей и архитектурные решения, обеспечивающие загрузку, асинхронную обработку, публикацию и воспроизведение видеоматериалов.

Цель второй главы заключается в разработке проектных решений, которые могут быть непосредственно использованы при программной реализации веб-системы. В отличие от аналитической части, где внимание уделялось особенностям видеостриминга и выбору технологического стека, здесь определяется состав UML-моделей, проектируется реляционная база данных, описываются интерфейсные экраны, алгоритмы обработки видео и общая архитектура приложения.

Проектирование системы ведётся с учётом уже выбранного технологического стека: серверного API на Go, очереди задач Redis, реляционной базы PostgreSQL, инструментов FFmpeg и Real-ESRGAN-ncnn-vulkan, а также SPA-клиента на Vue 3 с TypeScript. Такая связка позволяет заранее рассматривать не абстрактную схему, а конкретное инженерное решение, пригодное для последующей реализации и тестирования.

2.1 Моделирование информационной системы в нотации UML

Для формального описания проектируемой системы целесообразно использовать язык UML, позволяющий представить решение одновременно с

нескольких точек зрения: через сценарии использования, последовательность выполнения операций, состав программных сущностей и физическое размещение компонентов. Применение UML облегчает переход от сформулированных требований к конкретным проектным решениям и снижает риск рассогласования между пользовательскими сценариями и реализуемыми программными модулями.

В рамках данной работы UML-модели строятся для тех аспектов системы, которые являются критичными для видеосервиса: разграничение ролей, приём и обработка загрузок, асинхронное выполнение задач, публикация результата и потоковая доставка контента. Ниже приведены диаграммы, фиксирующие функциональное, структурное и архитектурное представление разрабатываемой системы.

2.1.1 Диаграмма вариантов использования

Диаграмма вариантов использования отражает состав внешних участников системы и набор действий, которые они выполняют в ходе повседневной работы с видеоматериалами. Для рассматриваемого проекта выделяются три основные роли: администратор, оператор контента и пользователь, просматривающий опубликованные материалы. Такой набор ролей соответствует выводам первой главы, где было показано, что задачи управления публикацией, обработки видеоданных и потребления контента должны быть разделены между разными категориями участников.

Администратор отвечает за инфраструктурный и организационный контур системы. В его полномочия входят управление учётными записями, настройка ролей и политик доступа, просмотр журналов событий, повторный запуск неуспешных заданий и контроль публикационной дисциплины. Администратор не выполняет массовую подготовку контента, однако именно он обеспечивает устойчивость системы и корректность жизненного цикла видеоматериалов.

Оператор контента является ключевым участником прикладного процесса. Он загружает исходные файлы через web-интерфейс, заполняет карточку видео, выбирает профиль обработки, при необходимости активирует AI-улучшение качества и отслеживает статус фоновых задач. После завершения обработки оператор публикует материал, корректирует метаданные или переводит ролик в архивное состояние. Таким образом, в его зоне ответственности находится большая часть сценариев, связанных с подготовкой контента к просмотру.

Пользователь взаимодействует только с уже опубликованной частью медиабазы. Он авторизуется в системе, осуществляет поиск и фильтрацию материалов, открывает страницу публикации и запускает воспроизведение в HLS-плеере. Для этой роли принципиальны простота навигации, устойчивость воспроизведения и наличие базовых средств поиска по каталогу.

Особое место занимают варианты использования, связанные не с отдельной ролью, а с целостностью процесса: авторизация и разграничение прав, запуск обработки, публикация или архивирование материала, а также ведение журналов и повторный запуск задач после ошибок. Именно эти сценарии связывают административный контур с контуром подготовки контента и обеспечивают единый жизненный цикл видеоданных внутри веб-системы.



Рисунок 2.1 – Диаграмма вариантов использования проектируемой системы

Таблица 2.1 – Роли пользователей и ключевые варианты использования

Роль	Основные варианты использования	Результат выполнения
Администратор	Авторизация, управление ролями, просмотр аудита, повторный запуск задач	Поддержание работоспособности и контролируемости системы
Оператор контента	Загрузка видео, заполнение карточки, запуск обработки, публикация	Подготовка видеоматериала к просмотру
Пользователь	Поиск каталога, открытие карточки, запуск плеера	Получение доступа к опубликованному контенту

Из диаграммы видно, что логика сервиса не ограничивается простым сценарием загрузки и просмотра файла. Даже для учебного проекта требуется поддержка ролевой модели доступа, прозрачного жизненного цикла публикации и механизмов управления фоновыми задачами. Это подтверждает, что разрабатываемая система ближе к специализированной информационной платформе, чем к обычному файловому хранилищу.

2.1.2 Диаграмма деятельности

Диаграмма деятельности используется для описания динамики ключевого процесса в системе. Для рассматриваемого видеосервиса таким процессом является загрузка исходного ролика и его последующая подготовка к потоковой публикации. В отличие от диаграммы вариантов использования, показывающей

состав ролей и их задачи, диаграмма деятельности позволяет проследить последовательность шагов и точки принятия решений внутри одного сквозного сценария.

Модель начинается с выбора файла и ввода метаданных, после чего система выполняет валидацию формата, размера и базовых технических характеристик. При успешной проверке создаётся карточка видео, а сама операция подготовки переносится в асинхронный контур. Далее worker-процесс запускает профиль FFmpeg, формирует HLS-сегменты и миниатюры, а при активированном AI-профиле дополнительно направляет материал на стадию улучшения качества. После завершения всех этапов запись переводится в статус готовности и становится доступной для публикации.

В диаграмме отдельно показаны ветви отказа. Если исходный файл не проходит проверку, система немедленно возвращает ошибку пользователю и не создаёт задание на обработку. Если же сбой возникает на фоне выполнения worker-задачи, то материал переводится в ошибочное состояние, а в журнале сохраняются технические сведения, необходимые для повторного запуска. Такая модель соответствует нефункциональным требованиям к надёжности и наблюдаемости системы.

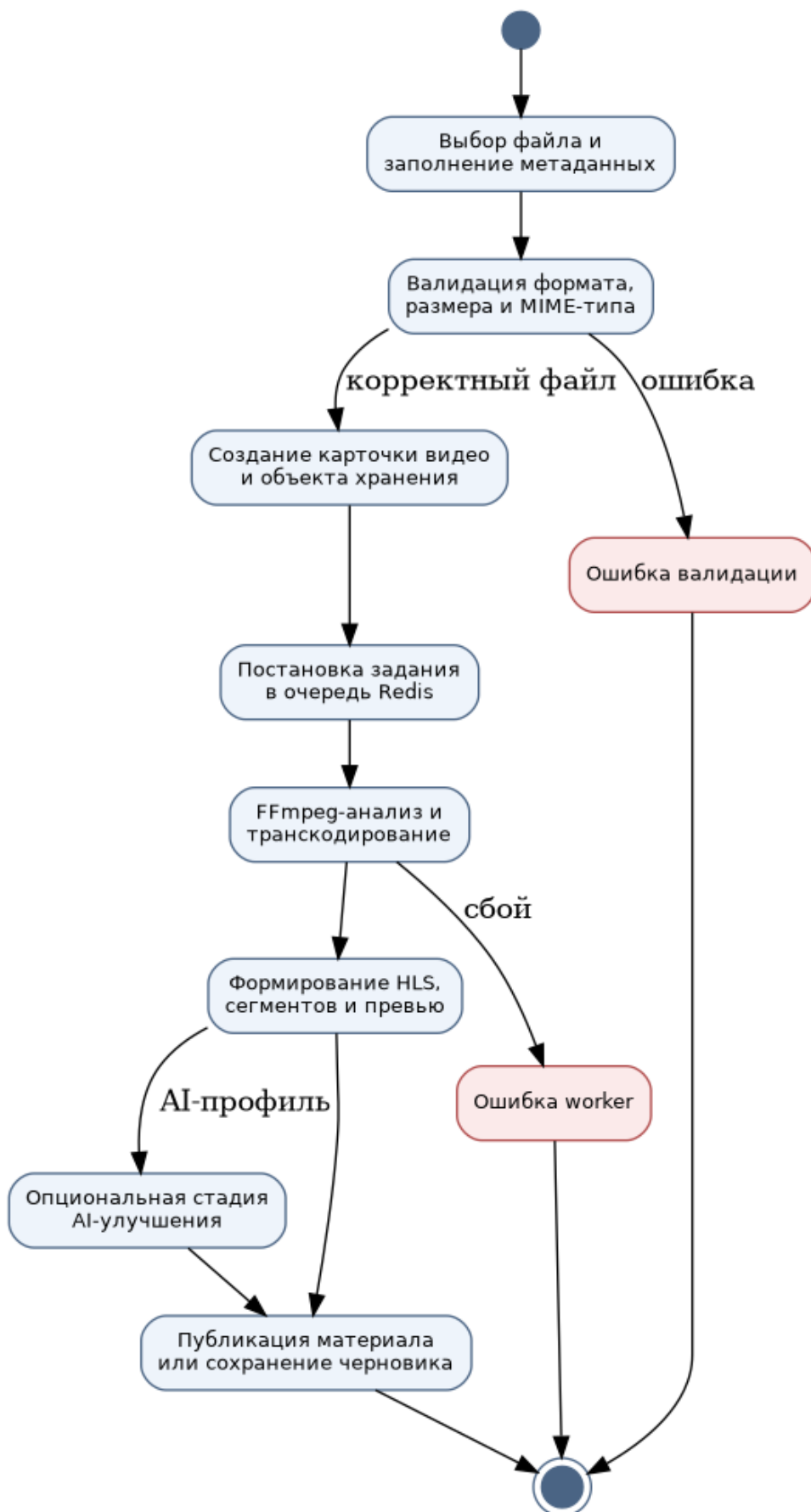


Рисунок 2.2 – Диаграмма деятельности процесса загрузки и подготовки видео

Представленная логика подчёркивает, что тяжёлые вычислительные операции не должны выполняться в контуре пользовательского HTTP-запроса. Разделение интерактивной и фоновой обработки уменьшает время отклика интерфейса и позволяет независимо масштабировать API и worker-процессы, что соответствует общим практикам построения высоконагруженных приложений [20].

2.1.3 Диаграммы последовательностей

Диаграммы последовательностей используются для уточнения обмена сообщениями между участниками и внутренними компонентами системы во времени. Для проектируемой веб-системы наиболее показательны два сценария: создание задания на обработку загруженного файла и воспроизведение уже опубликованного видеоматериала. Эти сценарии охватывают как контур публикации, так и контур потребления контента.

В первом сценарии пользовательский интерфейс передаёт файл и заполненную карточку через API. Серверная часть сохраняет оригинал в хранилище, регистрирует запись в базе данных и ставит задание в очередь Redis. После этого worker получает задачу, выполняет профиль FFmpeg, сохраняет результаты в хранилище и обновляет статусы обработки. Важным свойством такого сценария является асинхронность: пользователь получает подтверждение постановки задания до фактического завершения транскодирования.

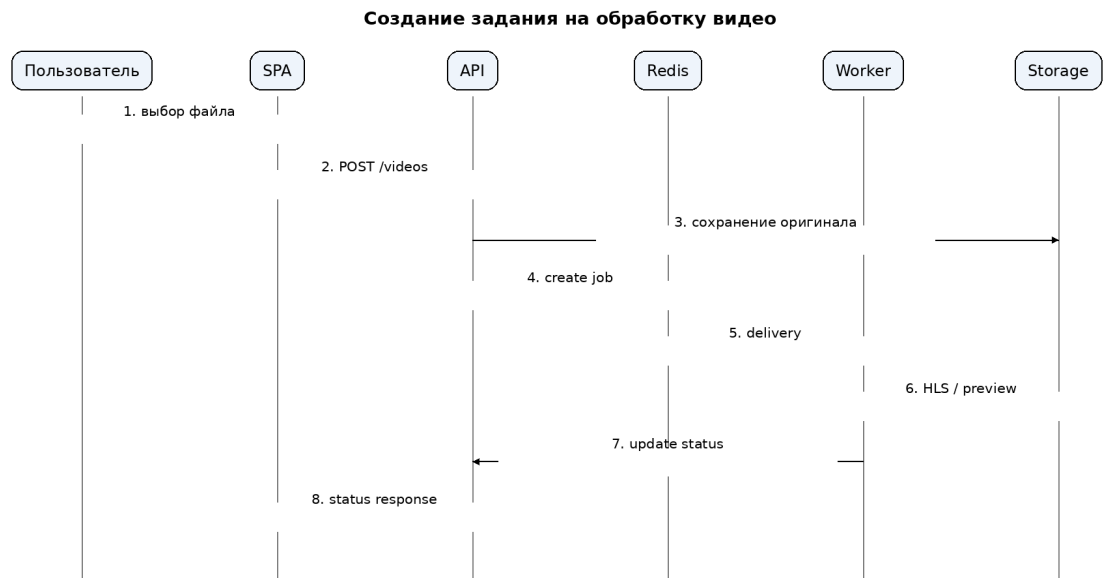


Рисунок 2.3 – Диаграмма последовательности создания задания на обработку

Во втором сценарии пользователь открывает карточку публикации, а SPA-приложение запрашивает через API метаданные и ссылку на манифест HLS. Сервер подготавливает модель страницы на основе базы данных и адресов объектов хранения, после чего браузер получает .m3u8-файл и обращается непосредственно к сегментам. Тем самым API участвует в принятии решения о доступности и составе публикации, но не становится узким местом для всей дальнейшей медиадоставки.

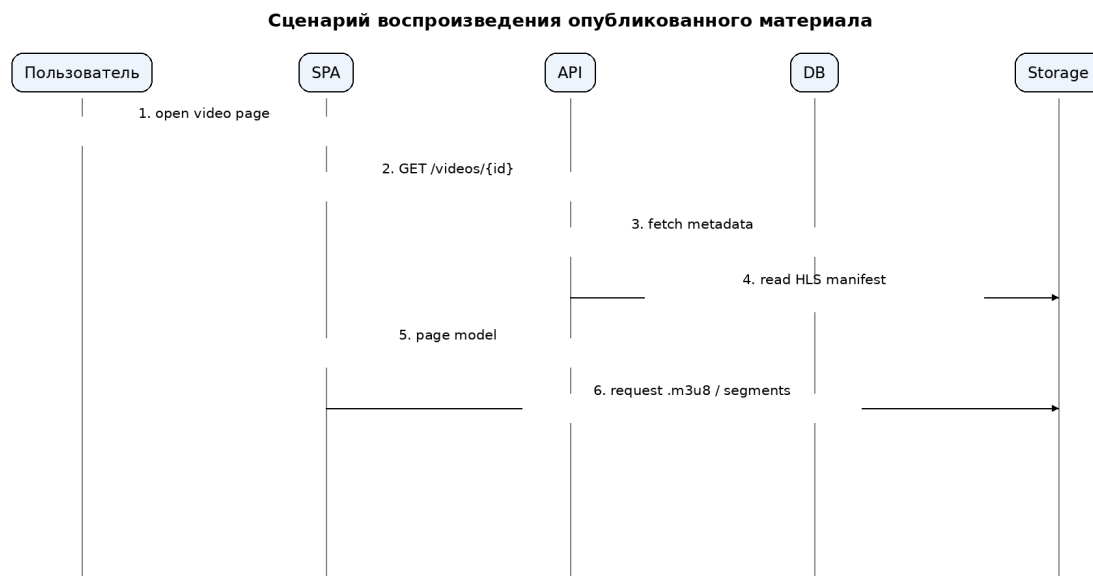


Рисунок 2.4 – Диаграмма последовательности воспроизведения
опубликованного видео

Использование диаграмм последовательностей позволяет заранее определить зоны ответственности отдельных компонентов. Обработка пользовательского ввода и визуализация результата относятся к SPA-клиенту, проверка правил и координация задач выполняются API-контуром, долговременное хранение и журналирование обеспечиваются PostgreSQL, а фактическая подготовка потоков переносится в worker-процессы.

2.1.4 Диаграмма классов

Диаграмма классов отражает статическую структуру проектируемого приложения и позволяет определить ключевые программные сущности, их атрибуты и связи. Для видеосервиса центральным объектом выступает класс «Видео», описывающий карточку материала, его статус, владельца и связь с исходным файлом в хранилище. С данным классом связаны сущности «Задание обработки», «Вариант потока», «Публикация», «Миниатюра» и «Журнал событий», формирующие вокруг видеоматериала полный операционный контур.

Класс «Пользователь» связан с классом «Роль», что обеспечивает разграничение прав между администраторами, операторами и конечными пользователями. Класс «Объект_хранилища» абстрагирует физические файлы и позволяет единообразно работать как с оригиналами, так и с манифестами, сегментами и превью. Такое разделение важно для дальнейшего переноса файлового слоя между локальным хранилищем и S3-совместимыми решениями без изменения прикладной модели данных.

Класс «Задание_обработки» фиксирует параметры конкретного запуска pipeline: тип операции, профиль кодирования, число попыток, статус и временные отметки начала и окончания. Это позволяет рассматривать обработку как самостоятельную сущность, а не как набор необязательных полей в карточке видео. Аналогично класс «Вариант_потока» описывает уже готовые представления различного качества, используемые HLS-плеером при адаптации к сетевым условиям.

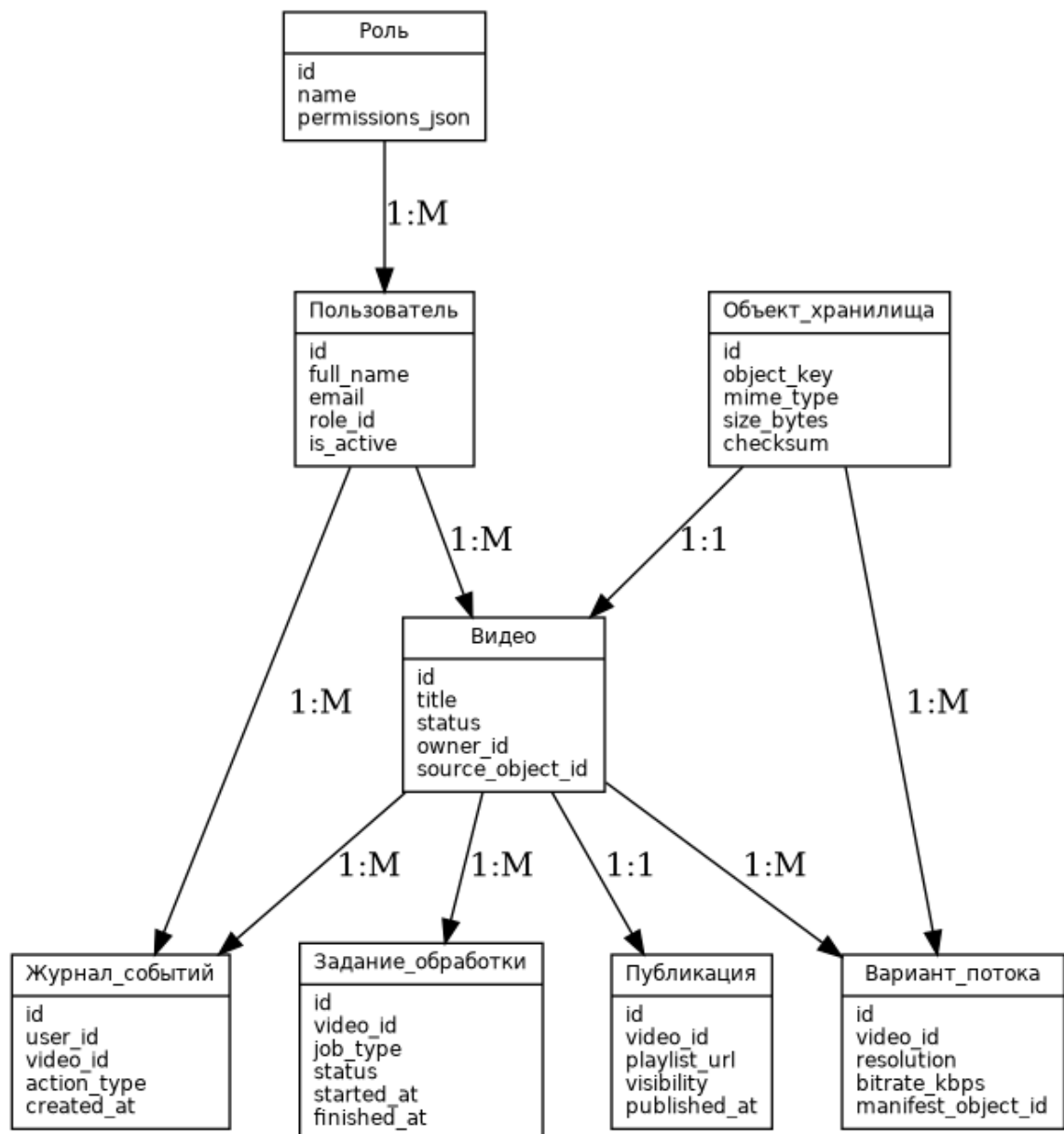


Рисунок 2.5 – Диаграмма классов проектируемой системы

Полученная статическая модель ориентирована на расширяемость. В дальнейшем в неё могут быть добавлены, например, сущности для плейлистов коллекций, внешних правил доступа или аналитики просмотров, не нарушая базовую схему связей вокруг класса «Видео». Это особенно важно для проекта, который из учебного прототипа должен иметь потенциал перехода к промышленному использованию.

2.1.5 Диаграмма компонентов

Диаграмма компонентов предназначена для представления крупных программных блоков и зависимостей между ними. Для рассматриваемой информационной системы структура модулей должна соответствовать выбранному в первой главе стеку и одновременно обеспечивать логическое разделение ответственности. В проекте целесообразно выделить SPA-интерфейс, API-контур, модуль каталога и публикации, оркестратор обработки, worker-процессы, слой работы с очередью Redis, реляционную базу данных и файловое либо объектное хранилище.

Такое разбиение позволяет изолировать пользовательский интерфейс от тяжёлой медиаподготовки. Модуль каталога работает с метаданными и карточками публикаций, в то время как модуль обработки координирует выполнение фоновых операций и не зависит от конкретного способа отображения контента в браузере. В результате каждая подсистема получает понятный контракт взаимодействия и может развиваться независимо, что соответствует принципам модульной архитектуры [20].

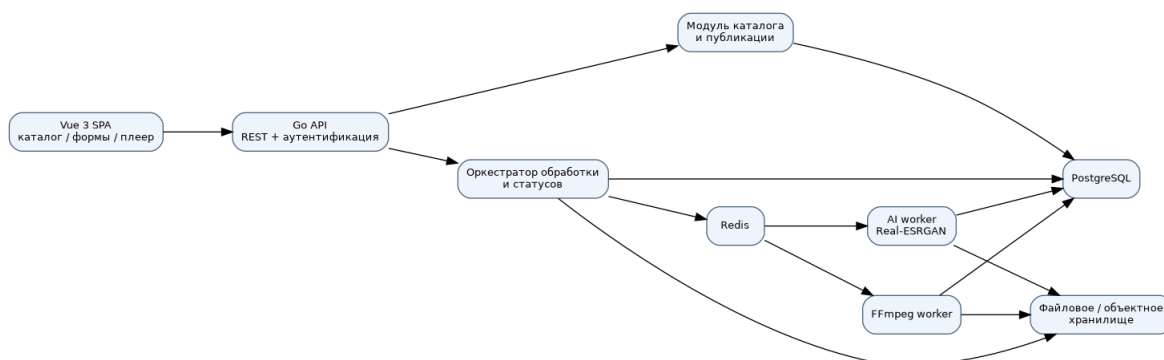


Рисунок 2.6 – Диаграмма компонентов проектируемой информационной системы

Диаграмма также подчёркивает, что интеграция с FFmpeg и AI-модулем должна выполняться через сервисы фоновой обработки, а не через прямые вызовы из web-контуров. Такой подход уменьшает связанность, облегчает

масштабирование и делает систему устойчивой к временной недоступности отдельных worker-узлов.

2.1.6 Диаграмма развертывания

Диаграмма развертывания показывает физическое размещение программных компонентов и каналы их взаимодействия. Для разрабатываемого сервиса целесообразно рассматривать конфигурацию, в которой браузер пользователя обращается к reverse проху, а тот маршрутизирует запросы к API-приложению. Отдельно располагаются PostgreSQL и Redis, а также контур фоновой обработки, состоящий из CPU-worker для FFmpeg и GPU-worker для AI-улучшения качества.

Файловые артефакты — оригиналы, HLS-манифесты, сегменты и миниатюры — вынесены в отдельное хранилище. Это позволяет независимо масштабировать медиаданные и прикладной контур, а также упрощает резервное копирование. При небольшом объёме проекта отдельные компоненты могут быть размещены на одном сервере, однако логическая схема развёртывания с самого начала предусматривает возможность их разнесения по разным узлам [20].

Клиентский контур

Браузер пользователя
SPA + HLS-плеер

Сервер приложений

Nginx / reverse proxy

Go API

PostgreSQL

Redis

Контур обработки

GPU worker
Real-ESRGAN

CPU worker
FFmpeg

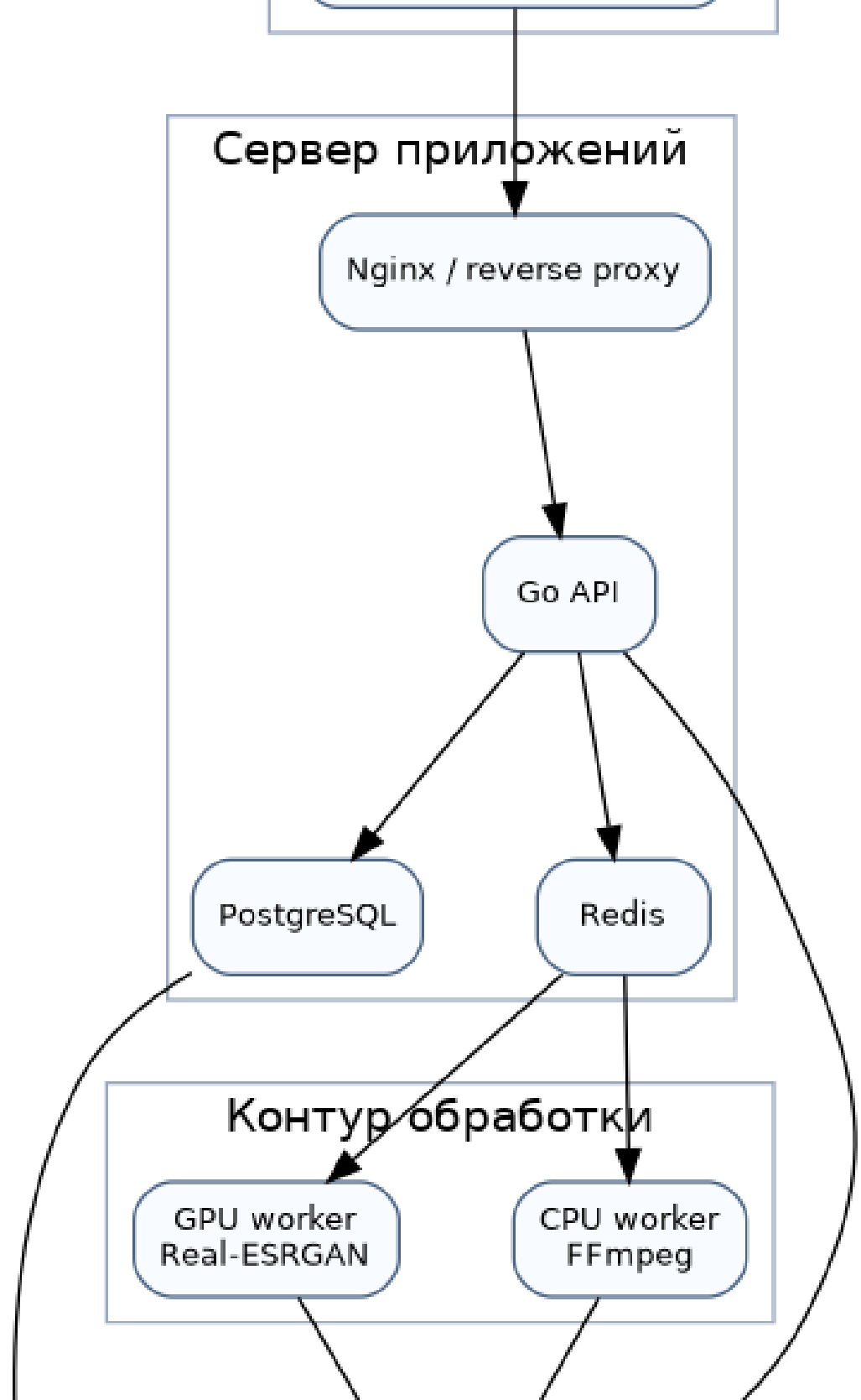


Рисунок 2.7 – Диаграмма развертывания проектируемой системы

Предложенная конфигурация соответствует задачам учебного проекта и одновременно обладает запасом для дальнейшего развития. Увеличение числа пользователей может быть компенсировано масштабированием API и worker-процессов, а подключение CDN или внешнего объектного хранилища не потребует пересмотра логики прикладных модулей.

2.2 Проектирование базы данных информационной системы

Хранение сведений о видеоматериалах, заданиях обработки и публикациях должно быть реализовано в реляционной базе данных, обеспечивающей целостность связей, поддержку конкурентных операций и удобство последующей фильтрации по каталогу. С учётом выбранного технологического стека в качестве целевой СУБД принимается PostgreSQL, обладающая развитой поддержкой транзакций, ограничений целостности и полуструктурированных полей [6], [21].

При проектировании структуры данных важно отделить прикладную метаинформацию от собственно бинарных медиаданных. Оригиналы файлов, сегменты HLS и превью хранятся в файловом либо объектном хранилище, тогда как база данных содержит только карточки видео, ссылки на объекты хранения, статусы обработки, параметры публикации и журналы операций. Такое разделение снижает нагрузку на СУБД и соответствует архитектурному решению, принятому в первой главе.

Логическая схема проектируемой базы данных приведена на рисунке 2.8. В центре модели находится сущность «Видео», с которой связаны владелец, исходный объект хранения, задания на обработку, готовые потоковые представления, публикации, миниатюры и записи аудита. За счёт этого обеспечивается полный жизненный цикл материала: от момента загрузки до архивирования.

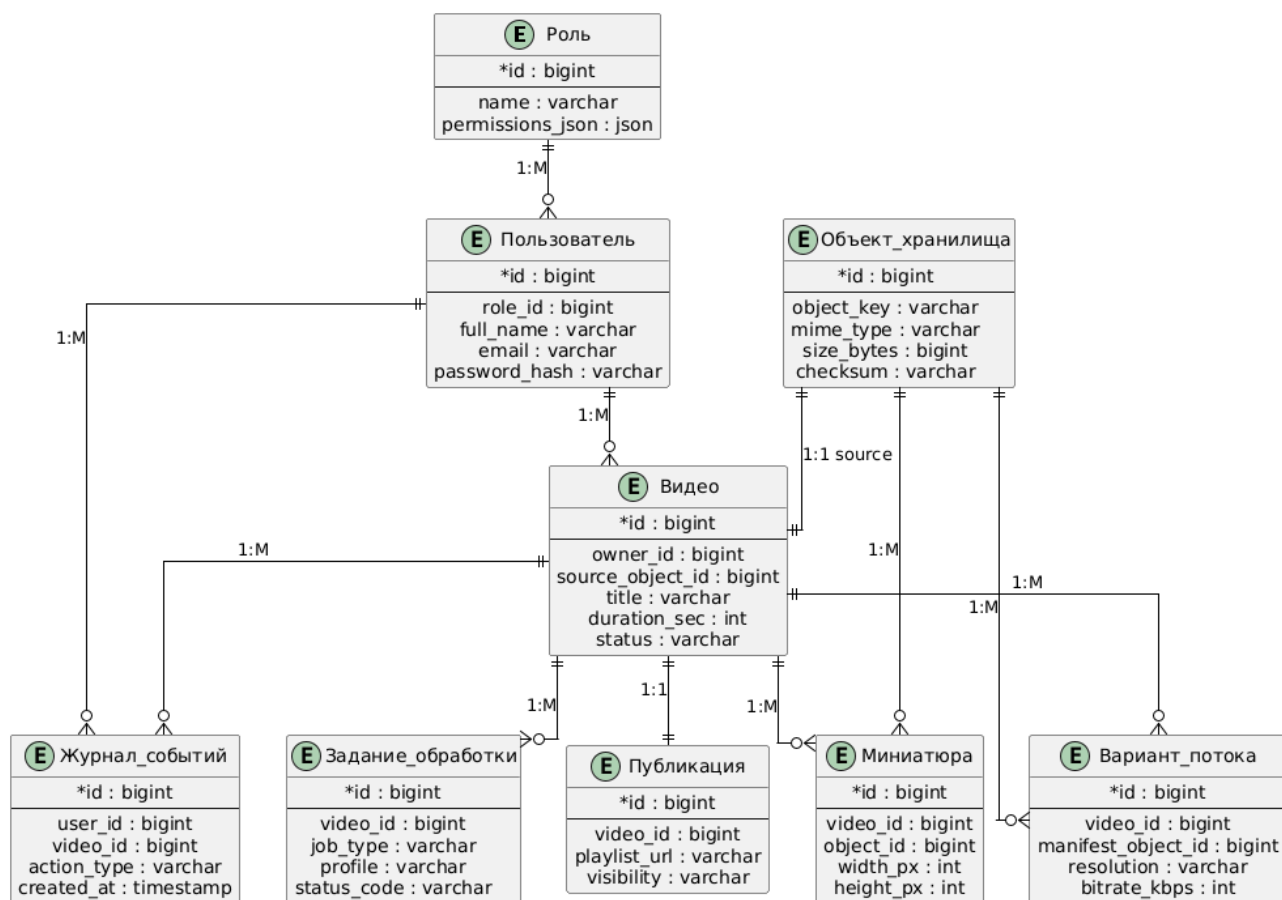


Рисунок 2.8 – Логическая схема базы данных видеосервиса

Таблица 2.2 – Основные сущности базы данных

Сущность	Назначение	Ключевые связи
Роль	Определение набора полномочий пользователя	1:M с пользователями
Пользователь	Учёт операторов, администраторов и зрителей	M:1 с ролью; 1:M с видео
Видео	Карточка видеоматериала и его текущий статус	M:1 с пользователем; 1:M с заданиями
Объект_хранилища	Связь логической сущности с физическим файлом	1:1 с видео; 1:M с вариантами потока
Задание_обработки	Фиксация запуска pipeline и его результата	M:1 с видео
Вариант_потока	Готовое представление определённого качества	M:1 с видео

Публикация	Параметры видимости и адрес HLS-манифеста	1:1 с видео
Миниатюра	Превью и изображения карточек	М:1 с видео
Журнал_событий	Аудит действий пользователей и фоновых процессов	М:1 с пользователем; М:1 с видео

На концептуальном уровне перечисленные сущности образуют ядро предметной области. Таблицы ролей и пользователей обеспечивают ролевую модель доступа; таблицы видео и объектов хранения описывают материальную основу системы; таблицы заданий, потоковых вариантов и публикаций фиксируют результат обработки; а таблица журнала событий поддерживает наблюдаемость и аудит критичных операций.

Ниже приведены структуры ключевых таблиц, обеспечивающих хранение и обработку данных в проектируемой системе.

Таблица «Роль» определяет состав системных полномочий и позволяет централизованно управлять доступом к административным и операторским функциям.

Таблица 2.3 – Структура таблицы «Роль»

Наименование поля	Идентификатор поля	Тип поля	Длина	Примечание
Ид_роли	id	BIGSERIAL	8	первичный ключ
Наименование	name	VARCHAR	50	обязательное поле
Описание	description	TEXT	—	назначение роли
Разрешения	permissions_json	JSONB	—	набор доступных операций
Системная	is_system	BOOLEAN	1	признак неизменяемой роли

Таблица «Пользователь» хранит сведения об учётных записях, необходимых для аутентификации, назначения ролей и связывания действий пользователя с конкретными видеоматериалами.

Таблица 2.4 – Структура таблицы «Пользователь»

Наименование поля	Идентификатор поля	Тип поля	Длина	Примечание
Ид_пользователя	id	BIGSERIAL	8	первичный ключ
ФИО	full_name	VARCHAR	150	обязательное поле
Электронная_почта	email	VARCHAR	120	уникальное поле
Хэш_пароля	password_hash	VARCHAR	255	обязательное поле
Ид_роли	role_id	BIGINT	8	внешний ключ
Активно	is_active	BOOLEAN	1	статус учётной записи
Дата_создания	created_at	TIMESTAMP	8	дата регистрации

Ключевая таблица «Видео» описывает карточку видеоматериала, его владельца, исходный объект хранения и текущее состояние обработки или публикации.

Таблица 2.5 – Структура таблицы «Видео»

Наименование поля	Идентификатор поля	Тип поля	Длина	Примечание
Ид_видео	id	BIGSERIAL	8	первичный ключ

Ид_владельца	owner_id	BIGINT	8	внешний ключ на пользователя
Ид_исходного_объекта	source_object_id	BIGINT	8	внешний ключ на объект хранения
Название	title	VARCHAR	200	обязательное поле
Описание	description	TEXT	–	карточка публикации
Длительность_сек	duration_sec	INTEGER	4	результат анализа файла
Статус	status	VARCHAR	30	uploaded / processing / published / failed
Опубликовано	is_published	BOOLEAN	1	флаг доступности
Дата_создания	created_at	TIMESTAMP	8	время загрузки

Таблица «Объект_хранилища» обеспечивает отделение физического файла от прикладной сущности и позволяет единообразно работать с оригиналами, манифестами, сегментами и миниатюрами.

Таблица 2.6 – Структура таблицы «Объект_хранилища»

Наименование поля	Идентификатор поля	Тип поля	Длина	Примечание
Ид_объекта	id	BIGSERIAL	8	первичный ключ
Ключ_объекта	object_key	VARCHAR	55 2	путь в хранилище
Тип_хранилища	storage_type	VARCHAR	0 3	local / s3
MIME_тип	mime_type	VARCHAR	00 1	тип содержимого
Размер_байт	size_bytes	BIGINT	8	размер файла
Контрольная_сумма	checksum	VARCHAR	28 1	целостность

Дата_создания	created_at	TIMESTAMP	8	время регистрации
---------------	------------	-----------	---	-------------------

Таблица «Задание_обработки» описывает каждую фоновую операцию pipeline и содержит сведения, необходимые для мониторинга, повтора и диагностики сбоев.

Таблица 2.7 – Структура таблицы «Задание_обработки»

Наименование поля	Идентификатор поля	Тип поля	Длина	Примечание
Ид_задания	id	BIGSERIAL	8	первичный ключ
Ид_видео	video_id	BIGINT	8	внешний ключ
Тип_операции	job_type	VARCHAR	40	analyze / transcode / enhance
Код_профиля	profile_code	VARCHAR	40	набор параметров кодирования
Статус	status	VARCHAR	30	queued / processing / done / failed
Число_попыток	attempts	INTEGER	4	для retry-механизма
Начато	started_at	TIMESTAMP	8	время старта
Завершено	finished_at	TIMESTAMP	8	время окончания
Фрагмент_лога	log_excerpt	TEXT	—	диагностика ошибок

Результат подготовки адаптивного потока хранится в таблице «Вариант_потока», где каждому представлению соответствует собственный набор параметров качества и ссылка на манифест либо префикс сегментов.

Таблица 2.8 – Структура таблицы «Вариант_потока»

Наименование поля	Идентификатор поля	Тип поля	Длина	Примечание
-------------------	--------------------	----------	-------	------------

Ид_варианта	id	BIGSERIAL	8	первичный ключ
Ид_видео	video_id	BIGINT	8	внешний ключ
Разрешение	resolution	VARCHAR	20	360p / 480p / 720p / 1080p
Битрейт_кбит_с	bitrate_kbps	INTEGER	4	скорость потока
Кодек	codec	VARCHAR	30	например h264
Ид_манифеста	manifest_object_id	BIGINT	8	ссылка на объект хранения
Префикс_сегментов	segment_prefix	VARCHAR	255	каталог сегментов
Основной	is_default	BOOLEAN	1	вариант по умолчанию

Таблица «Публикация» описывает параметры доступа к уже готовому видеоматериалу, включая адрес плейлиста, режим видимости и дополнительные ограничения.

Таблица 2.9 – Структура таблицы «Публикация»

Наименование поля	Идентификатор поля	Тип поля	Длина	Примечание
Ид_публикации	id	BIGSERIAL	8	первичный ключ
Ид_видео	video_id	BIGINT	8	уникальный внешний ключ
URL_плейлиста	playlist_url	VARCHAR	255	адрес master.m3u8
Режим_видимости	visibility	VARCHAR	20	public / private / token
Токен_доступа	access_token	VARCHAR	128	допускается NULL
Дата_публикации	published_at	TIMESTAMP	8	время открытия доступа
Дата_истечения	expires_at	TIMESTAMP	8	опционально

Для отображения карточек каталога и страниц публикаций используются миниатюры, выделенные в отдельную сущность. Это упрощает кэширование и позволяет хранить несколько видов превью для одного материала.

Таблица 2.10 – Структура таблицы «Миниатюра»

Наименование поля	Идентификатор поля	Тип поля	Длина	Примечание
Ид_миниатюры	id	BIGSERIAL	8	первичный ключ
Ид_видео	video_id	BIGINT	8	внешний ключ
Ид_объекта	object_id	BIGINT	8	ссылка на изображение
Ширина_пикс	width_px	INTEGER	4	размер превью
Высота_пикс	height_px	INTEGER	4	размер превью
Тип	kind	VARCHAR	20	poster / preview / hover
Дата_создания	created_at	TIMESTAMP	8	время формирования

Журнал событий фиксирует операции пользователей и фоновых процессов, что необходимо как для аудита, так и для последующей аналитики качества сервиса.

Таблица 2.11 – Структура таблицы «Журнал_событий»

Наименование поля	Идентификатор поля	Тип поля	Длина	Примечание
Ид_события	id	BIGSERIAL	8	первичный ключ
Ид_пользователя	user_id	BIGINT	8	допускается NULL для system
Ид_видео	video_id	BIGINT	8	допускается NULL
Тип_действия	action_type	VARCHAR	50	код операции
Детали	details	TEXT	–	структурированное описание
IP_адрес	ip_address	VARCHAR	45	источник запроса

Время_действия	created_at	TIMESTAMP	8	момент регистрации
----------------	------------	-----------	---	--------------------

Проектируемая база данных должна дополняться индексами по наиболее востребованным полям: статусу видео, владельцу, времени создания задания, типу операции, а также разрешению и битрейту потокового варианта. Для предотвращения логических противоречий рекомендуется использовать ограничения уникальности для публикаций и внешние ключи для всех связей между сущностями. В совокупности это создаёт основу для устойчивой реализации прикладной логики и последующего программирования серверного контура.

Дополнительно можно предусмотреть частичные индексы по опубликованным материалам и полям полнотекстового поиска, связанным с названием, описанием и тегами. Это позволит ускорить работу пользовательского каталога и сделать отбор материалов по семантическим признакам более удобным.

2.3 Проектирование интерфейса информационной системы

При разработке интерфейса видеосервиса необходимо учитывать, что системой пользуются участники с различным уровнем подготовки: оператору нужен производственный контур загрузки и мониторинга, а конечному пользователю — простой каталог и устойчивый плеер. Поэтому интерфейс должен обеспечивать быстрое выполнение типовых действий, не перегружать экран второстепенной информацией и сохранять единый визуальный язык на всех страницах приложения.

Пользовательская часть проектируется как SPA-приложение на основе Vue 3, TypeScript и Vite [8], [9], [10]. Такой подход позволяет разделить интерфейс на независимые компоненты: форму авторизации, страницы каталога, карточки публикации, панель загрузки, экран статусов и административные разделы. Для

всех экранов применяются единые принципы: верхняя навигационная панель, основной контентный блок, компактные служебные панели и явная визуализация статусов обработки.

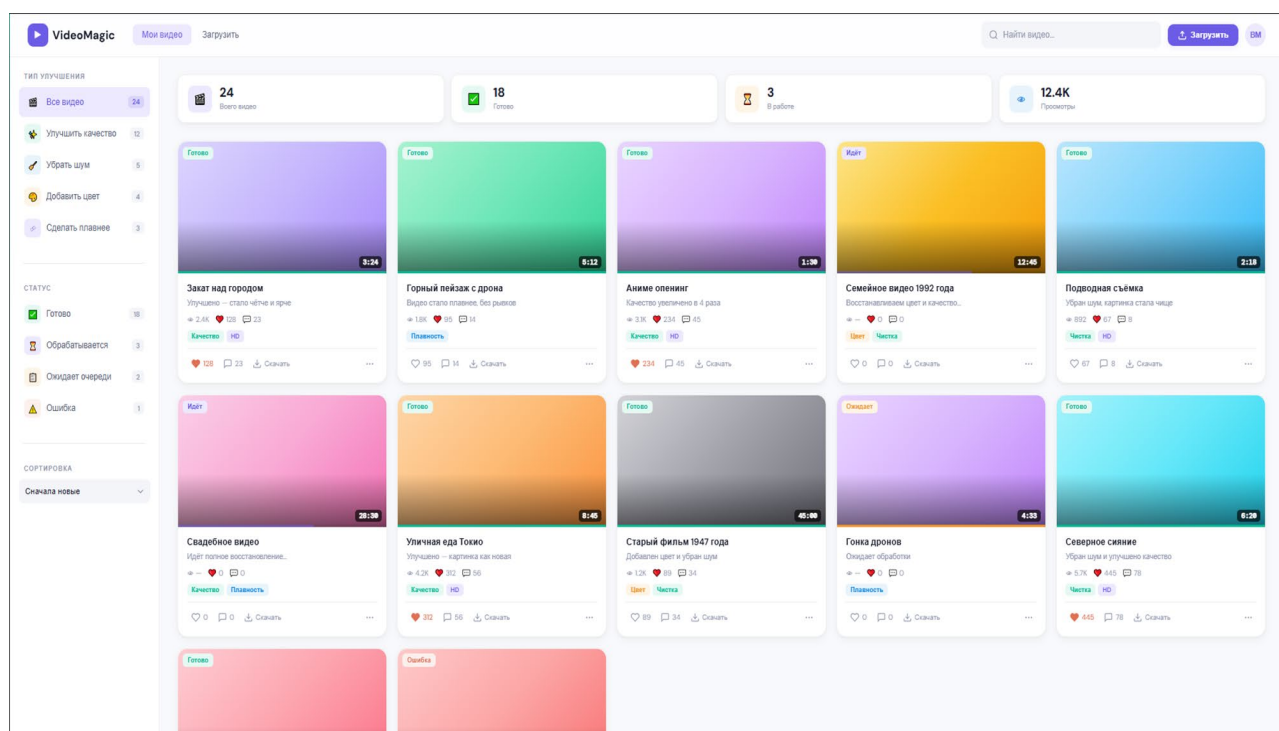


Рисунок 2.9 – Концептуальный макет главной страницы каталога

На главной странице каталога целесообразно разместить строку поиска, блоки фильтрации по категориям и тегам, сетку карточек публикаций и нижнюю область для плеера либо краткого описания выбранного материала. Такое расположение обеспечивает пользователю быстрый доступ к контенту без лишних переходов между разделами и делает каталог основной точкой входа в сервис.

Для оператора контента важна другая композиция экрана: слева должна находиться форма загрузки и редактирования карточки, а справа — панель статусов, очередь фоновых задач и журнал последних операций. Это позволяет выполнять подачу материалов в систему и одновременно контролировать выполнение pipeline. При проектировании такого экрана особенно важно избегать перегруженных интерфейсов: основные действия должны быть

сгруппированы по логике процесса, а долгие операции — сопровождаться заметными индикаторами.

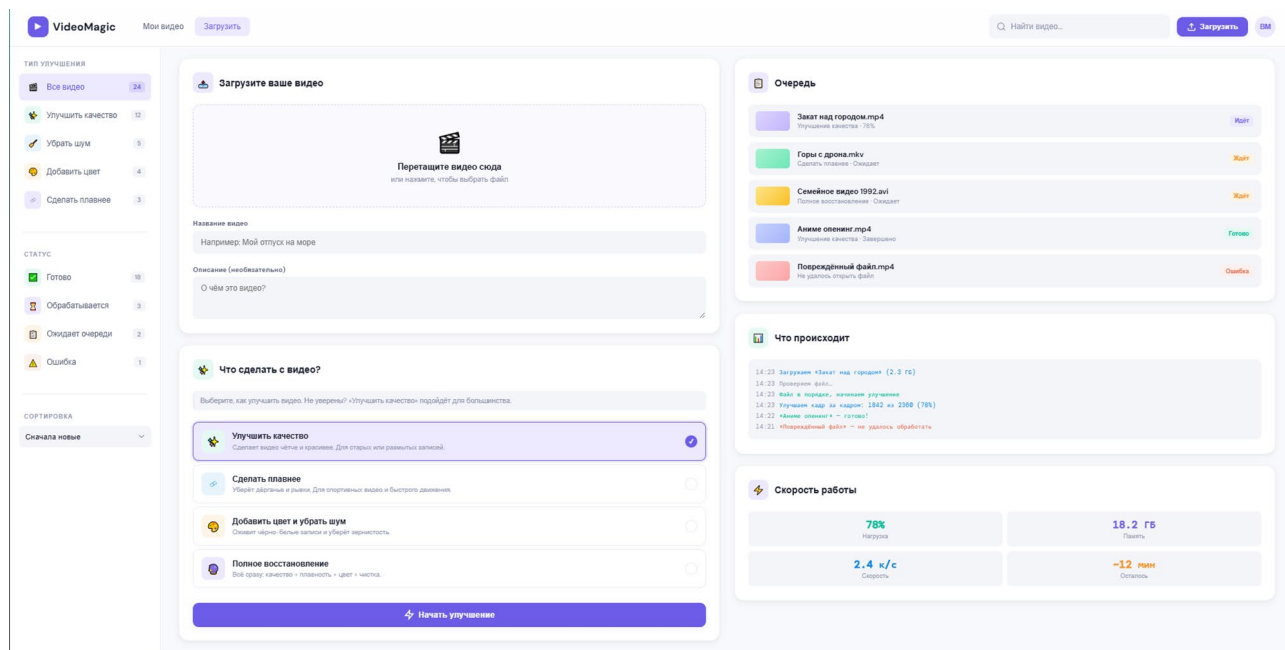


Рисунок 2.10 – Концептуальный макет страницы загрузки и мониторинга

Таблица 2.12 – Ключевые интерфейсные разделы системы

Раздел интерфейса	Основное содержимое	Основные роли
Страница авторизации	Форма входа, сообщения об ошибках, ссылки на восстановление	Все роли
Каталог публикаций	Поиск, фильтры, карточки видео, навигация по категориям	Пользователь, оператор
Страница видео	Плеер, описание, теги, техническая информация, выбор качества	Пользователь, оператор
Загрузка и редактирование	Поле выбора файла, карточка метаданных, профиль обработки	Оператор
Мониторинг обработки	Очередь задач, статусы, журнал ошибок, кнопки повтора	Оператор, администратор
Администрирование	Роли, пользователи, правила доступа, системные параметры	Администратор

Отдельного внимания требует экран публикации с встроенным HLS-плеером. Помимо самого окна воспроизведения, он должен содержать название ролика, описание, набор тегов, сведения о длительности и доступных качествах, а также служебные индикаторы, показывающие состояние воспроизведения и текущий поток. Для пользователя именно эта страница становится основным носителем ценности системы, поэтому её композиция должна быть лаконичной и не отвлекать от контента.

С точки зрения адаптивности интерфейс должен корректно работать на настольных и мобильных устройствах. Для небольших экранов часть вспомогательных панелей может сворачиваться, а навигация переводиться в компактный режим. Однако даже в мобильной версии ключевые сценарии — поиск публикаций, открытие карточки видео и запуск плеера — должны оставаться доступными без скрытой или многоуровневой навигации.

Практика проектирования SPA-интерфейсов показывает, что для административных экранов особенно важна наглядная цветовая дифференциация состояний: например, `queued`, `processing`, `failed` и `published`. Поэтому в пользовательском интерфейсе следует предусмотреть небольшие, но контрастные статусные индикаторы, не перегружающие экран и в то же время позволяющие оператору быстро считывать состояние задач.

2.4 Описание алгоритмов функционирования информационной системы

Работоспособность разрабатываемой информационной системы определяется совокупностью алгоритмов, реализующих приём файла, фоновую обработку, публикацию и последующее воспроизведение. В рамках проектирования особенно важны те алгоритмы, которые затрагивают длительные медиапроцессы и состояние карточки видео. Для них требуется не только корректная последовательность действий, но и явная фиксация переходов между статусами.

Базовый алгоритм начинается с загрузки ролика. После выбора файла интерфейс передаёт данные на сервер, где выполняется проверка MIME-типа, расширения, размера и доступности свободного места в хранилище. При успешной проверке создаётся карточка видео и запись о физическом объекте хранения, после чего материал переводится в состояние `uploaded`. Далее сервер ставит задание на обработку в очередь `Redis` и возвращает оператору идентификатор созданной сущности.

Таблица 2.13 – Основные статусы обработки видеоматериала

Статус	Условие установки	Следующее допустимое состояние
<code>uploaded</code>	исходный файл принят и карточка создана	<code>validating</code> / <code>queued</code>
<code>validating</code>	выполняется первичный анализ файла	<code>queued</code> / <code>failed</code>
<code>queued</code>	задание помещено в очередь <code>Redis</code>	<code>processing</code>
<code>processing</code>	<code>worker</code> выполняет транскодирование или AI-этап	<code>published</code> / <code>enhanced</code> / <code>failed</code>
<code>enhanced</code>	AI-обработка завершена успешно	<code>published</code>
<code>published</code>	материал доступен для просмотра	<code>archived</code>
<code>failed</code>	обнаружена ошибка файла или <code>pipeline</code>	<code>queued</code> (при повторе)
<code>archived</code>	материал снят с публикации	<code>published</code>

Алгоритм фоновой подготовки включает анализ метаданных, выбор профиля кодирования и запуск команд `FFmpeg` [3]. На этой стадии система формирует несколько потоковых представлений разного качества, создаёт HLS-манифест и набор сегментов фиксированной длительности. Если пользователь активировал AI-улучшение, то перед финальной публикацией выполняется дополнительная стадия суперразрешения, основанная на `Real-ESRGAN-ncnn-vulkan` [11], [12]. Поскольку данный этап существенно более ресурсоёмок, он

должен запускаться как отдельная ветка pipeline и не блокировать базовую транскодирующую логику.

После завершения обработки алгоритм публикации связывает карточку видео с master.m3u8, записывает параметры доступа и переводит материал в состояние published. При обнаружении ошибки система формирует диагностическое сообщение, сохраняет фрагмент технического лога и переводит задачу в состояние failed, что даёт оператору возможность повторно запустить обработку без повторной загрузки исходника.

Алгоритм воспроизведения начинается с запроса карточки публикации. Клиент получает адрес HLS-манифеста, после чего браузерный плеер самостоятельно выбирает подходящий вариант качества в зависимости от доступной скорости сети и объёма буфера [1]. С точки зрения сервиса важно, что логика адаптации переносится на клиентскую сторону, а сервер отвечает за корректность заранее подготовленных представлений и доступность файлов сегментов.

Таким образом, алгоритмы функционирования системы объединяют синхронные и асинхронные операции, а также обеспечивают явный контроль состояний. Это делает проектируемый сервис устойчивым к временным сбоям и пригодным для масштабирования, поскольку каждый этап pipeline может быть выделен в независимый программный модуль.

Для повышения надёжности рекомендуется сохранять в базе не только финальные статусы, но и служебные метки прогресса: проценты транскодирования, идентификатор worker-узла, факт запуска AI-этапа и сведения о повторных попытках. Эти данные не обязательно должны отображаться конечному пользователю, однако они ценны для эксплуатации, мониторинга и технической поддержки.

2.5 Архитектура информационной системы

Архитектура разрабатываемой информационной системы должна обеспечивать согласованную работу пользовательского интерфейса, серверного API, реляционного хранилища метаданных, очереди фоновых задач и средств медиаподготовки. С учётом результатов первой главы наиболее целесообразной является многокомпонентная клиент-серверная архитектура, в которой интерактивные операции отделены от ресурсоёмких фоновых вычислений. Такой подход соответствует общим практикам построения высоконагруженных приложений и упрощает горизонтальное масштабирование отдельных узлов [20].



Рисунок 2.12 – Архитектурная схема взаимодействия компонентов системы

Клиентский контур реализуется в виде SPA-приложения на Vue 3 с типобезопасным доступом к REST-эндпоинтам. Серверная часть на Go + Gin отвечает за аутентификацию, работу с карточками видео, регистрацию объектов хранения, публикацию, поиск по каталогу и выдачу служебных данных для

интерфейса. Метаданные и связи между сущностями сохраняются в PostgreSQL [6], а координация фоновых задач осуществляется через Redis [7].

Контур обработки включает CPU-worker для транскодирования и GPU-worker для AI-улучшения качества. Оба worker-модуля получают задания из очереди, забирают оригиналы из хранилища, формируют производные объекты и обновляют статусы в базе данных. Такой подход позволяет увеличивать производительность за счёт независимого масштабирования worker-узлов без изменения прикладной логики каталога или публикационного API.

Таблица 2.14 – Основные компоненты архитектуры и их ответственность

Компонент	Зона ответственности	Ключевые интеграции
Vue SPA	Каталог, формы загрузки, карточка публикации, плеер	REST API, HLS-плеер
Go API	Аутентификация, валидация, метаданные, публикация	PostgreSQL, Redis, хранилище
Redis	Очередь фоновых задач и буфер статусов	Go API, worker-модули
FFmpeg worker	Транскодирование, HLS-пакетирование, миниатюры	Redis, хранилище, PostgreSQL
AI worker	Опциональное улучшение качества видео	Redis, GPU/Vulkan, хранилище
PostgreSQL	Карточки видео, пользователи, статусы, аудит	Go API, worker-модули
Object storage	Оригиналы, сегменты, манифесты, превью	Go API, worker-модули, браузер

В архитектуре необходимо выделить и группы прикладных API. Отдельные наборы эндпоинтов используются для аутентификации, работы с карточками видео, управления загрузками, публикации результатов и мониторинга фоновых задач. Такое разделение упрощает сопровождение и позволяет в будущем выделить наиболее нагруженные функции в самостоятельные сервисы или версии API.

Таблица 2.15 – Основные группы REST-эндпоинтов системы

Группа эндпоинтов	Назначение	Примеры операций
Auth	Вход, выход, проверка роли	POST /login, GET /me
Videos	Карточки видеоматериалов и каталог	POST /videos, GET /videos/{id}, GET /videos
Uploads	Работа с исходными файлами и объектами хранения	POST /uploads, GET /uploads/{id}
Jobs	Мониторинг и повторный запуск обработки	GET /jobs, POST /jobs/{id}/retry
Publication	Публикация, архивирование, ссылки на HLS	POST /videos/{id}/publish, POST /videos/{id}/archive
Admin	Пользователи, роли, журналы событий	GET /users, GET /audit

С точки зрения безопасности архитектура должна предусматривать разграничение ролей, валидацию загружаемых файлов, защиту административных эндпоинтов и журналирование чувствительных операций. Для частных публикаций целесообразно использовать подписанные URL или токены доступа, а для административного контура — обязательную проверку роли и ведение аудита действий.

Для обеспечения надёжности необходимо сохранять состояние всех фоновых задач и иметь возможность безопасного повторного запуска. Сбой одного worker-узла не должен приводить к потере карточки видео или исходного файла. По этой причине критичными становятся идемпотентность операций, запись контрольных статусов в PostgreSQL и хранение промежуточных логов. Такой набор мер делает архитектуру устойчивой и удобной для сопровождения.

Предложенная архитектура допускает несколько сценариев развития. На начальном этапе система может работать на одном сервере с локальным хранилищем, а при росте объёма контента и числа пользователей отдельные

компоненты могут быть вынесены на самостоятельные узлы: PostgreSQL — на выделенный сервер, объектное хранилище — на S3-совместимый сервис, а AI-worker — на GPU-станцию. Благодаря модульной структуре такие изменения не потребуют полного пересмотра прикладного кода.

В целом предложенная архитектурная схема формирует баланс между производительностью, контролируемостью и возможностью развития. Она ориентирована не только на реализацию MVP, но и на поэтапное усложнение сервиса: добавление аналитики просмотров, политик доступа, интеграции с внешними каталогами и автоматизированных сценариев подготовки контента.

Выводы по второй главе

Во второй главе были сформированы основные проектные решения для информационной системы потоковой передачи видеоконтента. В нотации UML описаны пользовательские сценарии, логика ключевых процессов, состав программных сущностей, структура модулей и схема развертывания системы. Полученные модели подтверждают, что разрабатываемое решение охватывает полный цикл работы с видеоматериалом: от загрузки и постановки задания в очередь до публикации и потокового воспроизведения.

Кроме того, была спроектирована реляционная база данных, определены ключевые интерфейсные разделы, описаны алгоритмы функционирования и предложена многокомпонентная архитектура на основе Go, PostgreSQL, Redis, FFmpeg и Vue. Материалы данной главы создают основу для последующей программной реализации системы и перехода к третьей главе выпускной квалификационной работы.

3 Разработка информационной системы потоковой передачи видеоконтента

На данном этапе проектные решения, сформированные во второй главе, переводятся в плоскость практической реализации. В рамках разработки необходимо было связать реляционное хранение метаданных, web-интерфейс, серверное API, очередь фоновых задач и медиаконвейер в единую работоспособную систему, обеспечивающую загрузку, обработку, публикацию и воспроизведение видеоконтента.

Реализация выполнялась с использованием выбранного ранее технологического стека: серверная часть разрабатывалась на Go с применением Gin, хранение метаданных организовано в PostgreSQL, постановка и получение фоновых заданий реализованы через Redis, обработка видео выполнена средствами FFmpeg, а опциональное улучшение качества — через Real-ESRGAN-nncnn-vulkan. Пользовательский интерфейс проектировался как SPA-приложение на Vue 3 и TypeScript.

Цель главы заключается в описании практической структуры программных модулей, реализации клиентских и серверных сценариев, а также проверке работоспособности созданной информационной системы на наборе типовых прикладных операций.

3.1 Программная реализация базы данных и серверных модулей

Практическая реализация системы начинается с создания реляционной схемы хранения, в которой должны быть отражены как прикладные сведения о публикации, так и технические состояния фоновой обработки. Для этого в PostgreSQL была сформирована совокупность таблиц, покрывающих пользователей, карточки видеоматериалов, наборы тегов, задания на обработку, сформированные потоковые представления и журналы выполнения. Применение

отдельных сущностей для карточки видео и задания на обработку позволяет независимо управлять жизненным циклом публикации и жизненным циклом технической подготовки ролика.

При создании схемы особое внимание уделялось целостности связей и формализации переходов между состояниями. Для карточек видео фиксируются владелец, заголовок, описание, признаки публикации и выбранный профиль обработки. Для заданий используется отдельная сущность, в которой сохраняются тип запуска, текущее состояние, попытка обработки, путь к журналу, а также ссылки на сформированные артефакты. Такой подход упрощает повторный запуск задач и обеспечивает прослеживаемость всей истории изменений.

Таблица 3.1 – Основные таблицы реализованной базы данных

Таблица	Назначение	Ключевые поля
users	Хранение учетных записей и ролей пользователей системы	id, email, role, password_hash, is_active
videos	Карточки видеоматериалов и параметры публикации	id, owner_id, title, status, visibility, manifest_path
tags	Справочник тематических тегов и категорий	id, name, slug
video_tags	Связь материалов с тематическими признаками	video_id, tag_id
processing_jobs	Очередь и история фоновой обработки	id, video_id, kind, status, attempt, started_at
playback_variants	Сведения о сформированных потоковых представлениях	video_id, quality, bitrate, playlist_path
audit_log	Журнал административных и технических действий	id, actor_id, event_type, created_at

Серверная часть приложения реализована по модульному принципу. На уровне HTTP-контуров расположены обработчики запросов, принимающие входные параметры, проводящие базовую валидацию и передающие управление

сервисному слою. Сервисный слой инкапсулирует прикладные правила: проверку ролей, разрешённость публикации, формирование заданий на обработку, регистрацию повторного запуска и подготовку данных для пользовательского каталога. Доступ к PostgreSQL и Redis изолирован репозиториями и адаптерами, что уменьшает связанность модулей и упрощает последующее тестирование [20], [21].

Для авторизации используется отдельный контур аутентификации, формирующий сессию или токен доступа после проверки учетных данных. После входа в систему для каждого запроса дополнительно проверяется роль пользователя: административные функции доступны только администратору, работа с загрузкой и повторным запуском задач — администратору и оператору контента, а просмотр каталога и воспроизведение — всем авторизованным пользователям с соответствующими правами доступа.

Наиболее важным серверным сценарием является загрузка видео. Endpoint приёма файла получает multipart-запрос, выполняет проверку допустимого расширения, размера и наличия обязательных полей карточки. После этого сервер сохраняет исходный файл во временное или постоянное хранилище, создаёт запись о видеоматериале в таблице videos, формирует техническое задание в processing_jobs и помещает его в очередь Redis. Возврат ответа пользователю выполняется без ожидания завершения FFmpeg-конвейера, что позволяет сохранять отзывчивость интерфейса при длительных операциях.

Дополнительно были реализованы служебные endpoint-ы для получения журнала обработки, просмотра текущего состояния задания, публикации готового материала, перевода записи в архив, а также повторного запуска неудачной операции без повторной загрузки оригинала. В совокупности это формирует управляемый прикладной API-контур, пригодный как для браузерного клиента, так и для последующего расширения интеграциями с внешними системами.

Таблица 3.2 – Ключевые REST-endpoint-ы серверной части

Метод и маршрут	Назначение	Используется в разделе интерфейса
POST /api/auth/login	Аутентификация пользователя и выдача доступа	Экран авторизации
GET /api/videos	Получение каталога и фильтрация материалов	Каталог и поиск
POST /api/videos/upload	Загрузка исходного файла и создание карточки	Форма загрузки
GET /api/videos/{id}	Чтение карточки материала и статусов	Страница просмотра / мониторинг
POST /api/videos/{id}/publish	Публикация или снятие материала с публикации	Административные действия
POST /api/jobs/{id}/retry	Повторный запуск завершившегося ошибкой задания	Журнал и ошибки
GET /api/jobs/{id}/log	Получение журнала фоновой обработки	Мониторинг и статусы

3.2 Реализация клиентской части и пользовательских сценариев

Клиентская часть была реализована как одностраничное web-приложение, обеспечивающее переход между ключевыми пользовательскими сценариями без полной перезагрузки страницы. Такой подход особенно удобен для экранов загрузки и мониторинга, где требуется регулярно обновлять состояния фоновых задач, отображать прогресс обработки и оперативно реагировать на изменение статусов. Использование Vue 3 и TypeScript позволило структурировать интерфейс в виде независимых компонентов, отвечающих за формы ввода, карточки материалов, списки задач, диалоговые окна и панель проигрывателя [8], [9], [10].

Маршрутизация клиентского приложения разделяет публичные и защищённые разделы. Экран авторизации используется как входная точка в

систему; после успешной проверки учётных данных пользователь перенаправляется на доступный ему прикладной раздел. Для администратора и оператора доступна панель управления загрузкой, мониторингом и публикацией, тогда как для обычного пользователя основной сценарий сводится к поиску материалов в каталоге и просмотру подготовленных видеозаписей.

Интерфейс каталога строится вокруг фильтрации и быстрых действий с карточками материалов. Пользователь может отобразить записи по названию, тегам, статусу, владельцу и признаку публикации, после чего открыть подробную карточку видеоматериала. Для операторского режима дополнительно отображаются технический статус, наличие превью, список доступных вариантов качества и элементы управления повторной обработкой.

Отдельное внимание в клиентской части уделено экрану загрузки и мониторинга. Здесь объединяются форма выбора файла, заполнение карточки, задание профиля обработки, переключатель публикации и признаки AI-улучшения. Правая часть экрана предназначена для контроля очереди, просмотра журнала и обработки ошибок. Благодаря асинхронному обмену с API интерфейс может обновлять статусы задач, не блокируя пользователя на время выполнения FFmpeg-конвейера.

Страница просмотра использует web-плеер на основе HTML5-воспроизведения и HLS-манифеста. В зависимости от состояния материала пользователь видит либо готовый проигрыватель, либо сервисное сообщение о том, что ролик ещё обрабатывается. Для администратора и оператора на той же странице доступны сведения о выбранном профиле обработки, журнал фоновых операций, а также команды публикации, архивирования и повторного запуска.

Таблица 3.3 – Основные клиентские компоненты и их назначение

Компонент	Назначение	Повторное использование
AuthForm	Ввод учетных данных и передача запроса на аутентификацию	Страница авторизации
VideoCatalog	Фильтрация, сортировка и вывод карточек материалов	Каталог / административная панель
UploadForm	Выбор файла, ввод метаданных и отправка на сервер	Страница загрузки
StatusPanel	Отображение очереди, журнала и состояний обработки	Мониторинг задач
VideoPlayerPanel	Просмотр HLS-потока и карточки публикации	Страница просмотра
NotificationToast	Сообщения о результате операций и ошибках	Все прикладные экраны

3.3 Реализация конвейера обработки видеоконтента

После получения исходного файла система формирует асинхронный pipeline обработки. Сначала worker извлекает медиаметаданные при помощи ffprobe, определяя длительность, разрешение, кодек, параметры аудио и другие технические характеристики. Затем подготавливаются рабочие директории хранения и вычисляется набор действий, соответствующий выбранному профилю публикации. Именно на этом этапе определяется, требуется ли только базовое транскодирование или должна быть задействована ветка AI-улучшения.

Базовая ветка конвейера реализована средствами FFmpeg. С её помощью формируются производные представления нескольких качеств, создаются HLS-плейлисты и сегменты, генерируются обложки и миниатюры. Структурирование обработки по последовательным стадиям позволяет явно фиксировать текущий шаг выполнения, а при возникновении ошибки — регистрировать точку сбоя, чтобы не терять управляемость процесса и иметь возможность повторного запуска с понятной диагностикой [3].

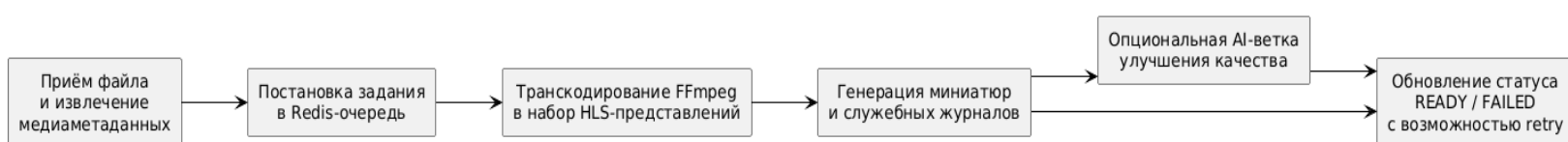


Рисунок 3.3 – Реализованный конвейер обработки видеоконтента

Для роликов низкого качества предусмотрена опциональная ветка AI-улучшения. Она запускается по выбору пользователя либо по административному правилу и использует Real-ESRGAN-ncnn-vulkan как отдельный этап фоновой обработки. Поскольку такая операция значительно тяжелее обычного транскодирования, система ограничивает количество одновременно исполняемых GPU-задач и фиксирует для них отдельные статусы, что исключает влияние AI-обработки на базовые сценарии публикации [11], [12].

Итогом работы pipeline является формирование набора артефактов, пригодных для выдачи в браузерный плеер: оригинал, HLS-манифест, сегменты разных качеств, миниатюры и при необходимости улучшенная версия ролика. Пути к созданным файлам сохраняются в базе данных, что даёт возможность гибко связывать прикладную карточку материала с конкретными техническими представлениями и не дублировать одни и те же сведения в нескольких местах.

Таблица 3.4 – Этапы реализованного конвейера обработки

Этап	Входные данные	Инструмент	Результат
Прием файла	multipart-запрос и карточка видео	API-сервис	Исходный файл и запись в БД
Анализ медиаданных	Исходный файл	ffprobe	Технические характеристики ролика

Постановка задания	Идентификатор видео и профиль	Redis	Job в очереди обработки
Транскодирование	Исходный файл и параметры качества	FFmpeg	Набор производных потоков
HLS-упаковка	Видеопредставления	FFmpeg	Манифест и сегменты
AI-улучшение	Кадры или поток низкого качества	Real-ESRGAN-ncnn-vulkan	Улучшенная версия материала
Завершение	Пути к артефактам и журнал	API + PostgreSQL	Статус READY или FAILED

Таблица 3.5 – Организация файлового хранения артефактов обработки

Каталог	Содержимое	Использование в системе
original/	Исходные загруженные файлы	Повторная обработка без повторной загрузки
hls/	Плейлисты и сегменты адаптивной выдачи	Воспроизведение в web-плеере
previews/	Миниатюры и обложки материалов	Каталог и карточки видео
logs/	Технические журналы фоновой обработки	Мониторинг и диагностика
ai/	Промежуточные и итоговые артефакты AI-улучшения	Опциональная ветка обработки

3.4 Тестирование и оценка работоспособности системы

Для подтверждения корректности реализованных решений был сформирован набор функциональных и интеграционных сценариев проверки. Испытания проводились на локальном стенде, включающем браузерный клиент, API-приложение, PostgreSQL, Redis и отдельный worker обработки. Такая конфигурация позволяет воспроизвести полный жизненный цикл материала: от авторизации и загрузки файла до формирования HLS-представления и публикации ролика в каталоге.

В процессе тестирования особое внимание уделялось тем операциям, которые являются критичными для практической пригодности системы: прием видеофайла, корректная смена статусов обработки, формирование манифеста и сегментов, разграничение ролей, просмотр журналов фоновых задач и повторный запуск обработки в случае ошибки. В дополнение к этому проверялась устойчивость интерфейса к длительным операциям, чтобы пользователь не терял контроль над системой при продолжительной обработке файла.



Рисунок 3.4 – Тестовый стенд проверки системы

Таблица 3.6 – Основные функциональные сценарии тестирования

Код	Проверяемый сценарий	Ожидаемый результат	Итог проверки
F1	Авторизация оператора и администратора	Переход в разрешенный раздел интерфейса	успешно
F2	Загрузка корректного видеофайла	Создание карточки и постановка job в очередь	успешно
F3	Загрузка неподдерживаемого формата	Отказ в приеме файла и сообщение об ошибке	успешно
F4	Формирование HLS-представления	Появление плейлиста и сегментов в хранилище	успешно
F5	Публикация готового видеоматериала	Появление записи в пользовательском каталоге	успешно
F6	Повторный запуск неудачного задания	Создание новой попытки обработки и обновление журнала	успешно

F7	Просмотр готового материала в плеере	Старт воспроизведения и доступность вариантов качества	успешно
----	--------------------------------------	--	---------

Таблица 3.7 – Проверка нефункциональных характеристик системы

Группа проверки	Содержание проверки	Полученный результат
Отказоустойчивость	Сохранение статуса задачи и журнала при сбое worker-процесса	Повторный запуск доступен без повторной загрузки
Ролевой доступ	Соккрытие административных действий для обычного пользователя	Неразрешённые функции недоступны в UI и API
Наблюдаемость	Доступ к журналам обработки и статусам этапов	Логи и состояния отображаются в интерфейсе
Масштабируемость	Возможность выделения отдельных worker-процессов	Архитектура допускает независимое масштабирование
Совместимость	Работа интерфейса в современных браузерах	Базовые сценарии корректно воспроизводятся

Проведённые проверки показывают, что реализованная система корректно поддерживает базовые сценарии загрузки, асинхронной обработки и публикации видеоконтента. Особое значение имеет то, что длительные операции не блокируют пользовательский интерфейс, а их ход прозрачно отображается через панель мониторинга и журнал выполнения. Это подтверждает пригодность выбранной архитектуры для прикладного сервиса потоковой передачи видео.

Точная скорость обработки в реальной эксплуатации будет определяться характеристиками сервера, параметрами роликов и долей задач с AI-улучшением. Однако уже на уровне реализованной структуры видно, что узкие места могут быть вынесены в отдельные worker-процессы, а контуры хранения, API и фоновой обработки масштабируются независимо друг от друга. Следовательно, система обладает необходимым запасом для дальнейшего развития и перехода к более насыщенным эксплуатационным сценариям.

Выводы по третьей главе

В третьей главе была описана практическая реализация информационной системы потоковой передачи видеоконтента. Рассмотрены программная структура базы данных и серверных модулей, организация клиентского SPA-интерфейса, реализация конвейера обработки видеоматериалов и порядок взаимодействия между API, очередями задач, медиасервисами и хранилищем.

Дополнительно выполнено описание набора тестовых сценариев, подтвердивших корректность ключевых пользовательских и технических операций: авторизации, загрузки файлов, фоновой обработки, публикации, просмотра и повторного запуска задач. Полученные результаты показывают, что разработанная система может служить основой для дальнейшего расширения и подготовки итоговых выводов по всей выпускной квалификационной работе.

ЗАКЛЮЧЕНИЕ

В ходе выполнения выпускной квалификационной работы была решена задача разработки информационной системы потоковой передачи видеоконтента, ориентированной на контролируемый цикл загрузки, обработки, публикации и просмотра видеоматериалов. Актуальность выбранной темы связана с тем, что прикладные образовательные, корпоративные и специализированные видеосервисы предъявляют повышенные требования к управляемости медиаконвейера, прозрачности обработки и возможности интеграции собственных механизмов повышения качества контента.

На этапе исследования предметной области были проанализированы особенности video-on-demand, методы адаптивной доставки видео, роль транскодирования и значение асинхронной фоновой обработки. Проведённое сравнение существующих платформ видеостриминга показало, что готовые публичные и коммерческие решения обеспечивают базовые возможности публикации, однако не дают полного контроля над внутренним конвейером и плохо подходят для встраивания специализированных алгоритмов улучшения качества. На этой основе были сформулированы функциональные и нефункциональные требования к разрабатываемой системе и обоснован выбор технологического стека, включающего Go, PostgreSQL, Redis, FFmpeg, Vue и модуль Real-ESRGAN-ncnn-vulkan.

Во второй главе были получены основные проектные результаты. Построены UML-модели, отражающие пользовательские сценарии, последовательность ключевых операций, состав программных сущностей и архитектурную декомпозицию решения. Разработана логическая модель базы данных, описаны интерфейсные экраны, алгоритмы жизненного цикла видеоматериала и целевая архитектура сервиса. Это позволило перейти от

анализа требований к формальной проектной модели, пригодной для практической реализации.

В третьей главе проект был переведён в плоскость программной реализации. Описаны структура серверных модулей, организация реляционного хранения метаданных, состав REST-эндпоинтов, реализация клиентского SPA-интерфейса и конвейера фоновой обработки видеоконтента. Практическая часть показала, что разработанная система поддерживает ключевые пользовательские и административные сценарии: авторизацию, загрузку видеофайлов, валидацию и постановку задач в очередь, формирование HLS-представлений, публикацию материалов, просмотр статусов обработки и повторный запуск неуспешных операций.

Проведённое тестирование подтвердило работоспособность выбранных архитектурных решений. Система корректно обрабатывает базовые функциональные сценарии и демонстрирует требуемые нефункциональные свойства: разграничение ролей, наблюдаемость, возможность повторной обработки без повторной загрузки исходного файла и готовность к независимому масштабированию отдельных контуров. Тем самым цель выпускной квалификационной работы, заключающаяся в повышении удобства и эффективности обработки видеоконтента за счёт создания веб-системы для его загрузки и улучшения качества, достигнута.

Практическая значимость полученного результата состоит в том, что предложенная система может служить основой для корпоративных видеоплатформ, образовательных порталов, закрытых медиатек и внутренних баз знаний. В дальнейшем разработка может быть расширена за счёт подключения объектного хранилища и CDN, добавления поддержки MPEG-DASH, развития аналитики просмотров, интеграции с корпоративными механизмами аутентификации и настройки более гибких профилей AI-улучшения. Таким образом, созданная информационная система обладает не только учебной

ценностью, но и потенциалом дальнейшего развития до прикладного промышленного решения.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1 Pantos R., May W. RFC 8216: HTTP Live Streaming. – URL: <https://www.rfc-editor.org/rfc/rfc8216> (дата обращения: 09.04.2026).

2 DASH Industry Forum. About MPEG-DASH. – URL: <https://dashif.org/about/> (дата обращения: 09.04.2026).

3 FFmpeg Documentation. – URL: <https://ffmpeg.org/documentation.html> (дата обращения: 09.04.2026).

4 The Go Programming Language Documentation. – URL: <https://go.dev/doc/> (дата обращения: 09.04.2026).

5 Gin Web Framework Documentation. – URL: <https://gin-gonic.com/en/docs/> (дата обращения: 09.04.2026).

6 PostgreSQL Documentation. – URL: <https://www.postgresql.org/docs/> (дата обращения: 09.04.2026).

7 Redis Documentation. – URL: <https://redis.io/docs/latest/> (дата обращения: 09.04.2026).

8 Vue.js Documentation. Introduction. – URL: <https://vuejs.org/guide/introduction.html> (дата обращения: 09.04.2026).

9 TypeScript Handbook. – URL: <https://www.typescriptlang.org/docs/handbook/intro.html> (дата обращения: 09.04.2026).

10 Vite Guide. Getting Started. – URL: <https://vite.dev/guide/> (дата обращения: 09.04.2026).

11 Wang X., Xie L., Dong C., Shan Y. Real-ESRGAN: Training Real-World Blind Super-Resolution with Pure Synthetic Data. – URL: <https://arxiv.org/abs/2107.10833> (дата обращения: 09.04.2026).

12 xinntao. Real-ESRGAN-ncnn-vulkan. – URL: <https://github.com/xinntao/Real-ESRGAN-ncnn-vulkan> (дата обращения: 09.04.2026).

13 YouTube Help. Upload YouTube videos. – URL: <https://support.google.com/youtube/answer/57407> (дата обращения: 09.04.2026).

- 14 Vimeo Developer. Working with Video Uploads. – URL: <https://developer.vimeo.com/api/upload/videos> (дата обращения: 09.04.2026).
- 15 JoinPeerTube. What is PeerTube? – URL: <https://joinpeertube.org/> (дата обращения: 09.04.2026).
- 16 PeerTube Documentation. Architecture. – URL: <https://docs.joinpeertube.org/contribute/architecture> (дата обращения: 09.04.2026).
- 17 Kinescope. Content Delivery Network with high-performance video processing. – URL: <https://kinescope.io/products/content-delivery-network> (дата обращения: 09.04.2026).
- 18 Kinescope. Video player. – URL: <https://kinescope.io/products/video-player> (дата обращения: 09.04.2026).
- 19 Таненбаум Э., Уэзеролл Д. Компьютерные сети. – 5-е изд. – СПб.: Питер, 2016. – 960 с.
- 20 Клеппманн М. Высоконагруженные приложения. Программирование, масштабирование, поддержка. – СПб.: Питер, 2018. – 640 с.
- 21 Коннолли Т., Бегг К. Базы данных: проектирование, реализация и сопровождение: теория и практика. – М.: Вильямс, 2017. – 1439 с.

ПРИЛОЖЕНИЯ

Приложение А – Исходный код серверной части

Приложение Б – Исходный код клиентской части

Приложение В – Схема базы данных

Приложение А

Исходный код серверной части

Листинг А.1 – Инициализация HTTP-сервера и маршрутов API

```
package main

import (
    "log"
    "net/http"
    "os"

    "github.com/gin-gonic/gin"
)

func main() {
    router := gin.Default()

    router.MaxMultipartMemory = 1024 << 20 // 1 GB

    api := router.Group("/api")
    {
        api.POST("/auth/login", loginHandler)
        api.POST("/videos/upload", uploadVideoHandler)
        api.GET("/videos/:id", getVideoHandler)
        api.GET("/videos", listVideosHandler)
        api.POST("/videos/:id/publish", publishVideoHandler)
        api.POST("/videos/:id/retry", retryProcessingHandler)
    }

    port := os.Getenv("PORT")
    if port == "" {
        port = "8080"
    }

    log.Printf("server started on port %s", port)
    if err := router.Run(":" + port); err != nil {
        log.Fatal(err)
    }
}

func loginHandler(c *gin.Context) {
    c.JSON(http.StatusOK, gin.H{"message": "login success"})
}
```

```

func getVideoHandler(c *gin.Context) {
    id := c.Param("id")
    c.JSON(http.StatusOK, gin.H{
        "id":    id,
        "title": "Example video",
        "status": "ready",
    })
}

func listVideosHandler(c *gin.Context) {
    c.JSON(http.StatusOK, gin.H{
        "items": []gin.H{
            {"id": 1, "title": "Video 1", "status": "ready"},
            {"id": 2, "title": "Video 2", "status": "processing"},
        },
    })
}

func publishVideoHandler(c *gin.Context) {
    c.JSON(http.StatusOK, gin.H{"message": "video published"})
}

func retryProcessingHandler(c *gin.Context) {
    c.JSON(http.StatusOK, gin.H{"message": "retry started"})
}

```

Листинг А.2 – Обработка загрузки видеофайла

```
package main

import (
    "net/http"
    "os"
    "path/filepath"

    "github.com/gin-gonic/gin"
)

func uploadVideoHandler(c *gin.Context) {
    file, err := c.FormFile("video")
    if err != nil {
        c.JSON(http.StatusBadRequest, gin.H{"error": "video file is required"})
        return
    }

    title := c.PostForm("title")
    description := c.PostForm("description")

    if title == "" {
        c.JSON(http.StatusBadRequest, gin.H{"error": "title is required"})
        return
    }

    uploadDir := "./storage/originals"
    _ = os.MkdirAll(uploadDir, os.ModePerm)

    dst := filepath.Join(uploadDir, filepath.Base(file.Filename))
    if err := c.SaveUploadedFile(file, dst); err != nil {
        c.JSON(http.StatusInternalServerError, gin.H{"error": "failed to save file"})
        return
    }

    videoID := 101

    // Здесь в реальном проекте:
    // 1. создаётся запись в БД
    // 2. извлекаются медиаметаданные
    // 3. создаётся задача в Redis-очереди

    c.JSON(http.StatusCreated, gin.H{
        "id":      videoID,
        "title":    title,
    })
}
```

```

        "description": description,
        "status":      "queued",
        "file_path":   dst,
    })
}

```

Листинг А.3 – Постановка задания в очередь обработки

```

package main

import (
    "context"
    "encoding/json"
    "log"

    "github.com/redis/go-redis/v9"
)

type ProcessingJob struct {
    VideoID      int    `json:"video_id"`
    SourcePath    string `json:"source_path"`
    Profile       string `json:"profile"`
    UseEnhancer   bool   `json:"use_enhancer"`
    OutputFolder string `json:"output_folder"`
}

func enqueueProcessingJob(rdb *redis.Client, job ProcessingJob) error {
    ctx := context.Background()

    payload, err := json.Marshal(job)
    if err != nil {
        return err
    }

    if err := rdb.RPush(ctx, "video:jobs", payload).Err(); err != nil {
        return err
    }

    log.Printf("job queued for video %d", job.VideoID)
    return nil
}

```

Листинг А.4 – Запуск FFmpeg для формирования HLS-представлений

```
package main

import (
    "fmt"
    "os/exec"
)

func runFFmpegToHLS(inputPath string, outputDir string) error {
    cmd := exec.Command(
        "ffmpeg",
        "-i", inputPath,
        "-preset", "fast",
        "-g", "48",
        "-sc_threshold", "0",
        "-map", "0:v:0",
        "-map", "0:a:0",
        "-c:v", "libx264",
        "-c:a", "aac",
        "-b:v", "2500k",
        "-b:a", "128k",
        "-f", "hls",
        "-hls_time", "4",
        "-hls_playlist_type", "vod",
        "-hls_segment_filename", fmt.Sprintf("%s/segment_%%03d.ts", outputDir),
        fmt.Sprintf("%s/master.m3u8", outputDir),
    )

    return cmd.Run()
}
```

Листинг А.5 – Запуск AI-улучшения качества через Real-ESRGAN

```
package main

import "os/exec"

func runRealESRGAN(inputImage string, outputImage string) error {
    cmd := exec.Command(
        "realesrgan-ncnn-vulkan",
        "-i", inputImage,
        "-o", outputImage,
        "-n", "realesrgan-x4plus",
        "-s", "2",
    )
```

```
)  
  
return cmd.Run()  
}
```

Приложение Б

Исходный код клиентской части

Листинг Б.1 – Маршрутизация SPA-приложения

```
import { createRouter, createWebHistory } from 'vue-router'  
import HomePage from '@pages/HomePage.vue'  
import UploadPage from '@pages/UploadPage.vue'  
import VideoPage from '@pages/VideoPage.vue'  
import AdminPage from '@pages/AdminPage.vue'  
  
const routes = [  
  {  
    path: '/',  
    name: 'home',  
    component: HomePage,  
  },  
  {  
    path: '/upload',  
    name: 'upload',  
    component: UploadPage,  
  },  
  {  
    path: '/videos/:id',  
    name: 'video',  
    component: VideoPage,  
    props: true,  
  },  
  {  
    path: '/admin',  
    name: 'admin',  
    component: AdminPage,  
  },  
]  
  
export const router = createRouter({  
  history: createWebHistory(),  
  routes,  
})
```

Листинг Б.2 – Компонент страницы загрузки видео

```
<script setup lang="ts">
import { ref } from 'vue'
import axios from 'axios'

const file = ref<File | null>(null)
const title = ref('')
const description = ref('')
const useEnhancer = ref(true)
const status = ref('')

const onFileChange = (event: Event) => {
  const target = event.target as HTMLInputElement
  if (target.files && target.files.length > 0) {
    file.value = target.files[0]
  }
}

const uploadVideo = async () => {
  if (!file.value || !title.value) {
    status.value = 'Выберите файл и укажите название'
    return
  }

  const formData = new FormData()
  formData.append('video', file.value)
  formData.append('title', title.value)
  formData.append('description', description.value)
  formData.append('useEnhancer', String(useEnhancer.value))

  try {
    const response = await axios.post('/api/videos/upload', formData)
    status.value = `Файл загружен, статус: ${response.data.status}`
  } catch (error) {
    status.value = 'Ошибка загрузки файла'
  }
}
</script>

<template>
<section class="upload-page">
  <h1>Загрузка видео</h1>

  <input type="file" accept="video/*" @change="onFileChange" />
  <input v-model="title" type="text" placeholder="Название видео" />
  <textarea v-model="description" placeholder="Описание"></textarea>
```

```

<label>
  <input v-model="useEnhancer" type="checkbox" />
  Включить AI-улучшение
</label>

<button @click="uploadVideo">Загрузить</button>

<p>{{ status }}</p>
</section>
</template>

```

Листинг Б.3 – Компонент каталога видеоматериалов

```

<script setup lang="ts">
import { onMounted, ref } from 'vue'
import axios from 'axios'

interface VideoItem {
  id: number
  title: string
  status: string
}

const videos = ref<VideoItem[]>([])

const loadVideos = async () => {
  const response = await axios.get('/api/videos')
  videos.value = response.data.items
}

onMounted(() => {
  loadVideos()
})
</script>

<template>
<section class="catalog-page">
  <h1>Каталог видео</h1>

  <div class="video-grid">
    <article v-for="video in videos" :key="video.id" class="video-card">
      <div class="video-card__preview"></div>
      <h2>{{ video.title }}</h2>
      <p>Статус: {{ video.status }}</p>
    </article>
  </div>

```

```
</section>
</template>
```

Листинг Б.4 – Воспроизведение HLS-видео в браузере

```
import Hls from 'hls.js'

export function initPlayer(videoElement: HTMLVideoElement, source: string) {
  if (Hls.isSupported()) {
    const hls = new Hls()
    hls.loadSource(source)
    hls.attachMedia(videoElement)
  } else if (videoElement.canPlayType('application/vnd.apple.mpegurl')) {
    videoElement.src = source
  }
}
```

Приложение В

Схема базы данных

Листинг В.1 – Пример схемы базы данных в SQL

```
CREATE TABLE roles (
  id BIGSERIAL PRIMARY KEY,
  name VARCHAR(50) NOT NULL UNIQUE,
  permissions_json JSONB
);

CREATE TABLE users (
  id BIGSERIAL PRIMARY KEY,
  role_id BIGINT REFERENCES roles(id),
  full_name VARCHAR(150) NOT NULL,
  email VARCHAR(120) NOT NULL UNIQUE,
  password_hash VARCHAR(255) NOT NULL,
  is_active BOOLEAN NOT NULL DEFAULT TRUE,
  created_at TIMESTAMP NOT NULL DEFAULT NOW()
);

CREATE TABLE storage_objects (
  id BIGSERIAL PRIMARY KEY,
  object_key VARCHAR(255) NOT NULL,
  mime_type VARCHAR(100),
  size_bytes BIGINT,
  checksum VARCHAR(128),
```

```

        created_at TIMESTAMP NOT NULL DEFAULT NOW()
    );

CREATE TABLE videos (
    id BIGSERIAL PRIMARY KEY,
    owner_id BIGINT NOT NULL REFERENCES users(id),
    source_object_id BIGINT REFERENCES storage_objects(id),
    title VARCHAR(200) NOT NULL,
    description TEXT,
    duration_sec INTEGER,
    status VARCHAR(30) NOT NULL DEFAULT 'uploaded',
    is_published BOOLEAN NOT NULL DEFAULT FALSE,
    created_at TIMESTAMP NOT NULL DEFAULT NOW()
);

CREATE TABLE processing_jobs (
    id BIGSERIAL PRIMARY KEY,
    video_id BIGINT NOT NULL REFERENCES videos(id),
    job_type VARCHAR(40) NOT NULL,
    profile_code VARCHAR(40),
    status VARCHAR(30) NOT NULL DEFAULT 'queued',
    attempts INTEGER NOT NULL DEFAULT 0,
    started_at TIMESTAMP,
    finished_at TIMESTAMP,
    log_excerpt TEXT
);

CREATE TABLE stream_variants (
    id BIGSERIAL PRIMARY KEY,
    video_id BIGINT NOT NULL REFERENCES videos(id),
    resolution VARCHAR(20) NOT NULL,
    bitrate_kbps INTEGER NOT NULL,
    codec VARCHAR(30),
    manifest_object_id BIGINT REFERENCES storage_objects(id),
    segment_prefix VARCHAR(255),
    is_default BOOLEAN NOT NULL DEFAULT FALSE
);

CREATE TABLE publications (
    id BIGSERIAL PRIMARY KEY,
    video_id BIGINT NOT NULL UNIQUE REFERENCES videos(id),
    playlist_url VARCHAR(255) NOT NULL,
    visibility VARCHAR(20) NOT NULL DEFAULT 'private',
    access_token VARCHAR(128),
    published_at TIMESTAMP,
    expires_at TIMESTAMP
);

```

```
CREATE TABLE thumbnails (  
  id BIGSERIAL PRIMARY KEY,  
  video_id BIGINT NOT NULL REFERENCES videos(id),  
  object_id BIGINT NOT NULL REFERENCES storage_objects(id),  
  width_px INTEGER,  
  height_px INTEGER,  
  kind VARCHAR(20),  
  created_at TIMESTAMP NOT NULL DEFAULT NOW()  
);  
  
CREATE TABLE event_logs (  
  id BIGSERIAL PRIMARY KEY,  
  user_id BIGINT REFERENCES users(id),  
  video_id BIGINT REFERENCES videos(id),  
  action_type VARCHAR(50) NOT NULL,  
  details TEXT,  
  ip_address VARCHAR(45),  
  created_at TIMESTAMP NOT NULL DEFAULT NOW()  
);
```